

JAVA | POO | UML | TESTES

POO na Prática com Blackjack

Um guia completo para transformar classes isoladas em um sistema de objetos coerente, testável e fácil de evoluir.

Apostila personalizada a partir do projeto **BlackJack**

Fundamentos, diagnóstico comentado, diagramas, exercícios e roteiro incremental

Edição de estudo | Agosto de 2026



COMO USAR ESTA APOSTILA

Aprender construindo, não copiando

Este material parte do seu código real. Ele explica os conceitos que causaram cada dúvida, mostra modelos mentais e exemplos pequenos, mas evita entregar uma solução completa pronta para copiar. O objetivo é que você consiga justificar cada atributo, método e relação antes de programá-los.

Regra de estudo

Leia um capítulo, feche o PDF e explique o conceito em voz alta. Depois faça somente o exercício correspondente no projeto. Se você não consegue prever o estado dos objetos após uma instrução, volte ao diagrama de referências.

O ciclo recomendado



Figura 1. Ciclo deliberado de aprendizagem usado ao longo do guia.

Convenções visuais

Elemento	Como interpretar
Conceito-chave	Uma ideia que deve entrar no seu modelo mental.
Alerta	Um erro comum ou uma armadilha observada no projeto.
Pergunta de professor	Uma pergunta para você responder antes de olhar o código.
Prática	Um exercício pequeno, com critério objetivo de conclusão.
Código	Um exemplo isolado; não é a solução completa do seu Blackjack.

Escopo do material

A apostila cobre todos os conceitos revelados pela versão atual do projeto: estado, referências, escopo, construtores, tipos, encapsulamento, coleções, imutabilidade, enums, responsabilidades, dependências, UML, fluxo do jogo, entrada de dados, testes e depuração.

NAVEGAÇÃO

Sumário

Parte 1: O modelo mental do Java	5
Onde seu projeto está agora	6
Classe, objeto, variável e referência	8
Escopo, campos, parâmetros e variáveis locais	10
Construtores, ordem de inicialização e invariantes	12
Tipos, contratos e categorias de erro	14
Parte 2: Estado protegido e responsabilidades	16
Encapsulamento, visibilidade e API pública	17
Coleções, generics, cópias e aliasing	20
static, instância e final	22
Responsabilidade única, coesão e acoplamento	24
Composição, herança e injeção de dependência	26
Enums, imutabilidade, identidade e igualdade	28
Parte 3: Modelando o domínio Blackjack	30
O grafo de objetos desejado	31
Projetando Carta: rank, naipe e pontuação	33
Projetando Baralho: gerar, embaralhar e entregar	35
Projetando Mão: estado, pontuação e ases	38
Participantes e fluxo completo da rodada	40
Interface textual, Scanner e controle de fluxo	43
Parte 4: UML e PlantUML com significado	45
PlantUML: da sintaxe ao modelo	46
Relações UML, direção e multiplicidade	48
Auditoria do blackjack.puml e evolução do modelo	50
Parte 5: Testes, diagnóstico e refatoração	52
O que torna um teste útil	53
Matriz de testes e casos de borda	56
Aleatoriedade, depuração e hipóteses	59
Refatoração segura e evolução incremental	61
Parte 6: Roteiro de prática deliberada	63
Dez marcos verificáveis	64

Exercícios em cinco níveis	66
Checklists de revisão do projeto	68
Autoavaliação e próximos desafios	70

Parte 7: Apêndices de consulta **72**

Auditoria da versão estudada	73
Referência rápida de Java	76
Referência rápida de PlantUML	78
Glossário essencial	80
Cartões de revisão e respostas conceituais	82
Regras simplificadas adotadas para a versão 1	83

PARTE 1

O modelo mental do Java

Antes de decidir métodos e relações, você precisa enxergar o que existe em tempo de execução: objetos, referências, escopos, estados e contratos de tipos.

NESTA PARTE

- Diagnóstico do projeto atual
- Classe, objeto e referência
- Atributo, parâmetro e variável local
- Construtores, null e invariantes
- Tipos e categorias de erro

CAPÍTULO 1

Onde seu projeto está agora

O código já tem bons substantivos, mas ainda se comporta como um conjunto de instruções soltas. O primeiro objetivo é distinguir o que compila, o que executa e o que realmente preserva estado.

OBJETIVOS DE APRENDIZAGEM

- Separar acertos estruturais de erros de lógica e modelagem.
- Ler o compilador como evidência, não como juiz de qualidade arquitetural.
- Transformar cada erro em uma pergunta que possa ser testada.

O que já está correto

- Você identificou as entidades centrais: **Carta**, **Baralho**, **Mao**, **Jogador**, **Dealer** e **Blackjack**.
- O baralho usa `List<Carta>` como tipo declarado e `ArrayList` como implementação. Essa é uma boa separação entre contrato e detalhe concreto.
- A chamada de `Blackjack` agora recebe um objeto `Jogador`, não mais o texto do nome. O projeto compila e o Maven conclui o teste.
- A expressão `this.nome = nome` demonstra corretamente a diferença entre campo e parâmetro.
- A interface de terminal está concentrada em `App`; isso facilita manter regras do jogo fora do código de entrada e saída.

O que compilar não garante

O compilador confirma que a sintaxe e os tipos usados são aceitáveis. Ele não consegue provar que uma mão continua existindo após o construtor, que cada carta conservou seu naipe, que o baralho não contém duplicatas ou que o jogo realmente iniciou. Essas são propriedades do comportamento e do modelo.

Camada	Pergunta	Exemplo no projeto
Compilação	O código respeita a linguagem e os tipos?	A chamada atual do construtor usa <code>Jogador</code> , portanto compila.
Execução	Alguma operação lança erro enquanto roda?	Entrada não numérica pode produzir <code>InputMismatchException</code> .
Lógica	O resultado é o esperado?	O nome tende a ser vazio ao misturar <code>nextInt()</code> e <code>nextLine()</code> .
Modelagem	A responsabilidade está no objeto coerente?	Uma única <code>Carta</code> fabricar treze cartas tem baixa coesão.

Retrato importante

Hoje, ao escolher `Blackjack`, o fluxo chega a criar 104 objetos `Carta`: 52 no baralho criado em `App` e outros 52 no inicializador do campo de `Blackjack`. O segundo baralho é substituído no construtor. Isso não é vazamento permanente, mas é trabalho inútil e revela uma estratégia de inicialização misturada.

Mapa dos sintomas atuais

Local	Sintoma observável	Conceito por trás
Mao	A lista nasce local e fica inalcançável.	Estado persistente e escopo.
Jogador / Dealer	A mão é criada, mas não é guardada.	Composição e referência de campo.
Carta	O naipe fica nulo e as figuras não são reconhecidas.	Invariantes e separação entre rank e pontuação.
Blackjack	Objetos são criados e logo substituídos.	Ordem de inicialização e injeção de dependência.
App	Nome vazio e menu incompleto.	Buffer de entrada e coerência de fluxo.
Teste	<code>assertTrue(true)</code> sempre passa.	Teste como experimento verificável.
PUML	Métodos e relações contradizem o código.	UML como modelo sincronizado.

Experimento 1 - diagnóstico sem editar

- 1 Para cada classe, escreva em uma folha quais dados deveriam continuar acessíveis após o construtor.
- 2 Marque onde cada dado está hoje: campo, parâmetro ou variável local.
- 3 Antes de executar, preveja o nome recebido pelo jogador após escolher a opção 1.
- 4 Explique o que o teste atual consegue comprovar e o que ele não consegue.

Critério de conclusão: Você consegue apontar uma evidência concreta para cada diagnóstico, sem usar apenas 'parece errado'.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Compilar é necessário, mas não suficiente. 2. Erros lógicos e de modelagem exigem invariantes e testes. 3. O diagnóstico deve apontar causa, efeito e forma de comprovação.	1. Qual propriedade do jogo o compilador não consegue verificar? 2. Que objeto é criado duas vezes durante a inicialização atual? 3. Por que um teste que nunca pode falhar não especifica comportamento?

CAPÍTULO 2

Classe, objeto, variável e referência

A frase 'criei uma mão' costuma esconder quatro conceitos diferentes. Separá-los é a mudança de modelo mental que mais evita confusão em POO.

OBJETIVOS DE APRENDIZAGEM

- Distinguir definição de classe, instância, variável e referência.
- Prever quando duas variáveis apontam para o mesmo objeto.
- Entender que atribuir uma referência não clona um objeto.



Figura 2. Três ideias distintas presentes em uma única linha de Java.

Dissecando uma declaração

EXEMPLO CONCEITUAL

```
Mao mao = new Mao();
```

Trecho	Papel
Mao	Tipo da variável: quais operações e valores são permitidos.
mao	Nome da variável de referência.
new Mao()	Construção de uma nova instância.
=	Cópia do valor da referência para a variável.

Analogia do endereço

A classe é um projeto de casa; o objeto é uma casa construída; a referência é um papel com o endereço. Dois papéis podem apontar para a mesma casa. A analogia é limitada: classes também definem comportamento, e a referência não deve ser tratada como um endereço físico acessível pelo programa.

Aliasing: dois nomes, um objeto

REFERÊNCIAS COMPARTILHADAS

```
Mao primeira = new Mao();
Mao segunda = primeira;
```

A segunda atribuição copia a referência, não a mão. Se uma carta for adicionada por meio de `segunda`, o estado visto por `primeira` também muda, pois existe apenas uma instância.

Situação	Quantidade de variáveis	Quantidade de objetos Mao	Consequência
<code>segunda = primeira</code>	2	1	Estado compartilhado intencional ou acidental.
Duas expressões <code>new Mao()</code>	2	2	Estados independentes.
Referência recebe <code>null</code>	1	0 alcançado	Nenhuma operação de instância pode ser chamada.

Java passa tudo por valor

Ao passar um objeto para um método, Java copia o *valor da referência*. O método recebe outra variável que aponta para a mesma instância. Ele pode modificar o objeto alcançado, mas reatribuir apenas o parâmetro não altera a variável do chamador.

Formulação precisa

Evite dizer 'Java passa objetos por referência'. Java passa argumentos por valor; quando o argumento é um objeto, o valor copiado é uma referência. Essa precisão explica por que mutar o objeto pode ser percebido fora do método, mas trocar o parâmetro local não troca o campo original.

Pare e preveja

- 1 Desenhe duas variáveis apontando para a mesma Mao.
- 2 Agora desenhe duas variáveis, cada uma recebendo uma expressão `new` diferente.
- 3 Se um método reatribui seu parâmetro para uma nova Mao, qual referência do chamador muda?
- 4 Explique por que compartilhar uma Mao entre jogador e dealer seria um bug.

Critério de conclusão: Seu desenho diferencia quantidade de referências e quantidade de instâncias.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Classe é definição; objeto é instância. 2. Variável de objeto guarda uma referência. 3. Atribuir referência não copia o objeto. 4. Java copia o valor dos argumentos.	1. Quantos objetos existem após duas chamadas a <code>new</code>? 2. Como duas variáveis podem observar a mesma mudança? 3. Qual é a diferença entre <code>null</code> e uma Mao vazia?

CAPÍTULO 3

Escopo, campos, parâmetros e variáveis locais

Um objeto só se lembra daquilo que está ligado ao seu estado. Declarar uma variável dentro do construtor não transforma essa variável em atributo.

OBJETIVOS DE APRENDIZAGEM

- Identificar o escopo de cada nome no código.
- Distinguir a vida da variável da alcançabilidade do objeto.
- Usar `this` para desambiguar campo e parâmetro.

Elemento	Onde é declarado	Até quando o nome é acessível	Uso típico
Campo de instância	Corpo da classe	Enquanto se possui uma referência para a instância	Estado próprio de cada objeto.
Parâmetro	Assinatura do método/construtor	Durante aquela chamada	Entrada recebida.
Variável local	Método, construtor ou bloco	Até o fim do bloco	Cálculo ou referência temporária.
Campo static	Corpo da classe com static	Associado à classe	Estado compartilhado, quando realmente necessário.

A autópsia da mão atual

MAO.JAVA - TRECHO ATUAL

```
public Mao() {  
    ArrayList<Carta> mao = new ArrayList<>();  
}
```

O nome `mao` é local ao construtor. Quando o construtor termina, o nome sai de escopo. Como a referência não foi copiada para nenhum campo, a lista fica inalcançável e elegível para coleta de lixo.

Não confunda variável e objeto

A variável local deixa de existir ao sair do escopo. O objeto não é necessariamente apagado naquele instante. Ele pode continuar vivo se outra referência alcançável apontar para ele; sem referência, fica elegível para o garbage collector, cujo momento de execução não é determinado.

Sombreamento e this

EXEMPLO FORA DO BLACKJACK

```
class Perfil {
    private String nome;

    Perfil(String nome) {
        this.nome = nome;
    }
}
```

O parâmetro `nome` sobrepõe o campo de mesmo nome dentro do construtor. `this` significa 'esta instância'. Assim, o lado esquerdo indica o campo permanente e o lado direito indica a entrada temporária.

A pergunta que decide

Pergunta de professor

Precisarei reencontrar este valor depois que o método terminar? Se sim, e se esse valor caracteriza o objeto, ele precisa estar alcançável por um campo. Se serve apenas para um cálculo intermediário, mantenha-o local.

Mapa de escopo do seu projeto

- 1 Sublinhe todos os campos das sete classes.
- 2 Circule todos os parâmetros de construtores e métodos.
- 3 Marque as variáveis locais que criam objetos e verifique se suas referências escapam do método.
- 4 Para cada lista, escreva quem será capaz de alcançá-la após a construção.

Critério de conclusão: Você consegue explicar por qual caminho de referências cada lista é acessada.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Campo representa estado; local representa trabalho temporário. 2. Sair de escopo não equivale a destruir imediatamente o objeto. 3. <code>this</code> aponta para a instância atual.	1. Por que a lista da <code>Mao</code> atual se perde? 2. O que <code>this.nome</code> identifica? 3. Quando uma variável temporária deve continuar local?

CAPÍTULO 4

Construtores, ordem de inicialização e invariantes

Construir não é executar a vida inteira do objeto. Um construtor deve estabelecer um estado válido e coerente para que os próximos métodos possam trabalhar com segurança.

OBJETIVOS DE APRENDIZAGEM

- Explicar a ordem básica de inicialização de uma instância Java.
- Definir invariantes para cada entidade do domínio.
- Distinguir criar dependências de recebê-las pelo construtor.

O que acontece durante `new Blackjack(...)`



Figura 3. Ordem conceitual relevante para compreender os objetos temporários do jogo.

BLACKJACK.JAVA - INICIALIZADORES ATUAIS

```
String nome;  
Baralho baralho = new Baralho();  
Jogador jogador = new Jogador(nome);  
Dealer dealer = new Dealer();
```

Quando `new Jogador(nome)` é executado, o campo `nome` ainda conserva seu valor padrão: `null`. Depois, o corpo do construtor substitui `baralho`, `jogador` e `dealer` pelos argumentos recebidos.

Invariantes: verdades permanentes

Uma invariante é uma condição que deve ser verdadeira logo após a construção e continuar verdadeira após toda operação pública. Ela transforma 'acho que está certo' em um contrato verificável.

Classe	Invariante inicial sugerida	Como poderia ser quebrada
Carta	Rank e naipe válidos e não nulos.	Não atribuir naipe ou aceitar texto arbitrário.
Mao	Coleção existente, vazia e sem null.	Criar lista somente como variável local.
Jogador	Nome válido e mão própria.	Aceitar nome vazio ou esquecer a mão.
Baralho	52 combinações únicas em um baralho novo.	Duplicar rank/naipe ou permitir alterações externas.

Classe	Invariante inicial sugerida	Como poderia ser quebrada
Blackjack	Baralho, jogador e dealer válidos.	Aceitar dependências nulas ou criar e substituir objetos.

Receber ou criar: escolha uma estratégia

Estratégia	Benefício	Custo
Objeto cria internamente	Uso simples e invariante local.	Menos controle para testes e configurações.
Objeto recebe pelo construtor	Dependências explícitas e facilmente substituíveis em testes.	O chamador precisa montar o grafo corretamente.

Injeção de dependência não exige framework

Quando App cria objetos e os fornece ao construtor de Blackjack, já existe injeção de dependência manual. O problema atual é criar objetos nos campos e depois receber outros objetos para substituí-los.

Ficha de invariantes

- 1 Para cada classe, complete: 'nasce válida quando...'.
- 2 Complete: 'pode ficar inválida se...'.
- 3 Decida quais dependências são intrínsecas ao objeto e quais devem ser fornecidas de fora.
- 4 Liste quais valores deveriam ser rejeitados no construtor, sem ainda programar a validação.

Critério de conclusão: Cada classe possui ao menos uma invariante observável por teste.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Construtor estabelece estado válido. 2. Inicializadores de campo executam antes do corpo do construtor. 3. Dependências devem ser criadas ou recebidas de modo coerente. 4. Invariantes orientam API e testes.	1. Por que o jogador temporário recebe nome null? 2. Que verdade deve valer para toda Carta válida? 3. O que torna uma dependência obrigatória explícita?

CAPÍTULO 5

Tipos, contratos e categorias de erro

O tipo não é apenas sintaxe: ele expressa o que uma operação espera receber e o que promete devolver. Bons contratos reduzem estados impossíveis antes mesmo da execução.

OBJETIVOS DE APRENDIZAGEM

- Ler a assinatura de um método como contrato.
- Diferenciar erro de compilação, execução, lógica e modelagem.
- Escrever pré-condições e pós-condições em linguagem natural.

A assinatura como fronteira

CONTRATO DO CONSTRUTOR

```
Blackjack(Baralho baralho, Jogador jogador, Dealer dealer)
```

Esse contrato exige três argumentos, nessa ordem e com tipos compatíveis. A correção recente em App substituiu o texto `nome` pelo objeto `user`, alinhando chamador e construtor.

Parte do contrato	Pergunta
Nome	Que intenção esta operação comunica?
Parâmetros	Quais entradas são necessárias?
Tipos	Que valores são aceitáveis em compilação?
Retorno	Que informação ou objeto volta ao chamador?
Pré-condição	O que precisa ser verdade antes da chamada?
Pós-condição	O que será verdade se a operação terminar normalmente?

Quatro falhas diferentes

Tipo de falha	Quando aparece	Exemplo
Compilação	Antes de executar.	Passar String onde o construtor exige Jogador.
Execução	Durante uma operação.	Ler texto com <code>nextInt</code> ou chamar método em <code>null</code> .
Lógica	Executa, mas produz efeito incorreto.	Guardar o nome vazio ou sobrescrever o valor calculado.
Modelagem	A estrutura não representa bem o domínio.	Carta fabricar um naipe inteiro ou Blackjack controlar a lista interna do baralho.

Compilador como parceiro

Não tente apenas silenciar a mensagem. Leia os tipos mencionados, encontre o contrato e compare com os argumentos reais. A mensagem costuma descrever o sintoma; sua tarefa é descobrir por que o modelo produziu aquela chamada incompatível.

Contrato em português antes do Java

Antes de escolher uma assinatura para retirar carta, escreva: 'dado um baralho não vazio, remover exatamente uma carta e devolver a mesma instância; depois, o tamanho será um menor'. Essa frase revela entrada implícita, efeito, retorno e tratamento necessário para baralho vazio.

Escreva contratos sem implementar

- 1 Descreva em português o contrato de uma mão receber carta.
- 2 Descreva o contrato de embaralhar sem afirmar que a ordem sempre mudará.
- 3 Descreva o contrato de calcular pontuação sem armazenar um total redundante.
- 4 Para cada contrato, indique uma pré-condição, uma pós-condição e um caso de erro.

Critério de conclusão: Outra pessoa conseguiria criar testes apenas lendo seus contratos.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Assinatura é parte do contrato, não o contrato inteiro. 2. Tipos removem classes inteiras de erro antes da execução. 3. Código compilável ainda pode estar logicamente ou conceitualmente errado.	1. Que tipo o construtor de Blackjack espera na segunda posição? 2. Qual é a pós-condição de retirar uma carta? 3. Como você distingue erro lógico de erro de modelagem?

PARTE 2

Estado protegido e responsabilidades

Com o modelo de referências estabelecido, o próximo passo é desenhar objetos que protegem suas invariantes, oferecem operações coerentes e colaboram sem conhecer detalhes demais.

NESTA PARTE

- Encapsulamento e visibilidade
- Coleções, generics e aliasing
- static, instância e final
- Coesão, acoplamento e delegação
- Composição, herança e dependências
- Enums, imutabilidade e igualdade

CAPÍTULO 6

Encapsulamento, visibilidade e API pública

Encapsular não é esconder tudo nem gerar getters e setters automaticamente. É controlar como o estado pode mudar para preservar as regras do objeto.

OBJETIVOS DE APRENDIZAGEM

- Distinguir esconder dados de proteger invariantes.
- Escolher uma superfície pública mínima e expressiva.
- Entender os quatro níveis de visibilidade usados em Java e UML.

Encapsulamento em uma frase

Conceito central

Um objeto encapsulado mantém seu estado sob controle e oferece operações que representam ações válidas do domínio. `private` é um mecanismo importante, mas a qualidade da API é o que realmente protege o modelo.

Imagine um caixa eletrônico. Ele não entrega a tabela de saldos para o cliente editar. Oferece ações como sacar, depositar e consultar. Do mesmo modo, uma mão não precisa entregar sua lista mutável; ela pode receber uma carta, informar quantidade e calcular pontuação.

Visibilidade

Java	PlantUML	Quem pode acessar	Uso típico
<code>public</code>	<code>+</code>	Qualquer classe com acesso ao tipo.	Operações que formam a API externa.
<code>private</code>	<code>-</code>	Somente a própria classe.	Estado interno e detalhes de implementação.
<code>protected</code>	<code>#</code>	Pacote e subclasses, com nuances.	Extensão planejada; use com cautela.
<code>package-private</code>	<code>~</code>	Classes do mesmo pacote.	Colaboração interna do pacote; ocorre quando não há modificador.

Detalhe observado

Os campos de `Blackjack` e o nome de `Jogador` não possuem modificador. Eles não são privados: têm visibilidade de pacote. Isso não causa erro de compilação, mas amplia a quantidade de código capaz de alterar o estado diretamente.

Getter não é sinônimo de encapsulamento

EXEMPLO PERIGOSO

```
public List<Item> getItens() {
    return itens;
}
```

Se `itens` é a lista interna mutável, o chamador pode limpar, reordenar ou inserir valores sem passar pelas regras da classe. O campo continua `private`, mas o controle foi perdido pela referência devolvida.

Forma de consulta	O que o chamador recebe	Observação
Informação específica	Número, booleano ou resultado calculado.	Menor exposição; geralmente comunica melhor a intenção.
Cópia da lista	Uma fotografia independente das referências.	Alterar a estrutura da cópia não altera a original.
Visão não modificável	Uma visão que rejeita mutação externa.	Pode refletir mudanças internas posteriores.
Lista original	A mesma estrutura mutável.	Quebra o controle e facilita estados inválidos.

Tell, Don't Ask

A ideia de 'diga, não pergunte' recomenda enviar uma intenção ao objeto que possui o estado, em vez de buscar seus dados, decidir tudo fora e gravar de volta. Não é uma lei absoluta, mas um bom sinalizador de responsabilidades vazando.

Forma frágil	Forma orientada ao domínio
Buscar lista do baralho e remover índice externamente.	Pedir ao baralho que entregue uma carta.
Buscar lista da mão e adicionar externamente.	Pedir à mão que receba uma carta.
Buscar todos os campos e calcular fora.	Pedir à mão sua interpretação de pontuação.

Auditoria da superfície pública

- 1 Liste o estado que cada classe deve proteger.
- 2 Para cada estado, invente uma alteração externa que poderia quebrá-lo.
- 3 Substitua mentalmente getters genéricos por perguntas ou ações do domínio.
- 4 Defina quais operações realmente precisam ser chamadas por outras classes.

Critério de conclusão: Sua API pública expressa intenções do Blackjack e não expõe estruturas internas mutáveis.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. <code>private</code> é necessário, mas não suficiente. 2. Encapsulamento protege invariantes por meio de operações válidas. 3. Ausência de modificador significa <code>package-private</code>. 4. Getter de coleção mutável pode furar o encapsulamento.	1. Por que devolver uma lista interna é perigoso? 2. Que operação comunica melhor 'retirar a próxima carta'? 3. Qual estado precisa permanecer exclusivo de <code>Mao</code>?

CAPÍTULO 7

Coleções, generics, cópias e aliasing

List garante o tipo dos elementos, mas não garante um baralho válido. A estrutura de dados e as invariantes do domínio resolvem problemas diferentes.

OBJETIVOS DE APRENDIZAGEM

- Distinguir interface List de implementação ArrayList.
- Entender o que generics garantem e o que não garantem.
- Reconhecer compartilhamento de listas e escolher cópias defensivas.

Abstração e implementação

DECLARAÇÃO TÍPICA

```
List<Carta> cartas = new ArrayList<>();
```

Parte	Significado
List	Contrato de uma sequência ordenada de elementos.
<Carta>	Parâmetro de tipo; o compilador espera referências compatíveis com Carta.
ArrayList	Implementação baseada em array redimensionável.
<>	Inferência do tipo genérico pelo compilador.

Analogia da tomada

List descreve o formato da tomada; ArrayList é um aparelho concreto compatível. O restante do programa depende do contrato e pode trocar a implementação quando houver motivo.

O que List não promete

- Não promete exatamente 52 elementos.
- Não impede cartas duplicadas.
- ArrayList aceita `null`.
- Não garante quatro naipes nem treze ranks.
- Não impede que outro objeto altere a mesma lista se a referência for compartilhada.

Generics protegem consistência de tipos em compilação. Regras como unicidade, limite e ausência de null são invariantes que a classe proprietária precisa estabelecer e testar.

addAll copia referências

Ao executar `cartas.addAll(outrasCartas)`, a lista de destino recebe referências para os mesmos objetos. As cartas não são clonadas. Isso é adequado quando `Carta` é imutável, mas precisa ser compreendido para evitar a expectativa de cópia profunda.

Cópia versus visão não modificável

Técnica conceitual	Estrutura independente?	Reflete mudanças futuras?	Pode conter null?
Nova <code>ArrayList</code> a partir da lista	Sim	Não	Sim
<code>List.copyOf</code>	Sim, estrutura imutável	Não	Não
<code>Collections.unmodifiableList</code>	Não; envolve a lista	Sim	Depende da original

final não congela a lista

Um campo `final List` impede que o campo seja reatribuído para outra lista. A própria lista ainda pode receber `add` e `remove`. Imutabilidade da referência, da coleção e dos elementos são três propriedades diferentes.

ArrayList e o topo do baralho

`ArrayList` remove rapidamente pelo fim. Remover o índice zero desloca os outros elementos; com 52 cartas o custo raramente importa, mas escolher uma extremidade consistente torna o contrato mais claro. Um `Deque` também poderia representar um monte, porém não é necessário introduzi-lo antes de dominar `List`.

Experimentos de identidade

- 1 Desenhe duas listas contendo referências para a mesma `Carta`.
- 2 Explique por que alterar uma `Carta` mutável seria visto pelas duas listas.
- 3 Explique por que remover a referência de uma lista não remove automaticamente da outra.
- 4 Liste três invariantes que `List` não garante sozinha.

Critério de conclusão: Você consegue separar estrutura da coleção, objetos armazenados e regras do domínio.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. <code>List</code> é contrato; <code>ArrayList</code> é implementação. 2. Generics garantem tipo, não validade do domínio. 3. Coleções guardam referências. 4. Cópia, visão e imutabilidade não são sinônimos.	1. O que <code>addAll</code> copia? 2. Por que <code>final List</code> ainda permite <code>add</code>? 3. Que risco existe ao expor a lista original?

CAPÍTULO 8

static, instância e final

static não significa 'mais fácil de chamar'. Significa que o membro pertence à classe e não a uma instância escolhida.

OBJETIVOS DE APRENDIZAGEM

- ☐ Decidir se um comportamento pertence a uma instância.
- ☐ Entender por que this não existe em contexto static.
- ☐ Distinguir referência final de objeto imutável.

Pergunta	Membro de instância	Membro static
A quem pertence?	A cada objeto individual.	À classe como um todo.
Existe this?	Sim, a instância atual.	Não.
Estado compartilhado?	Cada instância possui o seu.	Um único valor para todas.
Exemplo coerente	Embaralhar aquele baralho.	Ponto de entrada main ou fábrica sem estado.

A pergunta decisória

Pergunta de professor

A operação faz sentido sem escolher uma instância específica? Para embaralhar, é necessário saber qual baralho terá a ordem alterada. Isso aponta naturalmente para comportamento de instância do objeto que possui as cartas.

O método atual `Blackjack.embaralhar(Baralho)` é static, recebe um baralho e está vazio. Além de estar longe do estado que pretende modificar, ele não pode acessar diretamente a lista privada de `Baralho`. Uma API encapsulada faria o próprio objeto dono do estado executar a mudança.

Quando static é coerente

- `main`, porque a JVM precisa de um ponto de entrada sem construir App.
- Função pura ou utilitário realmente sem estado de instância, usado com moderação.
- Fábrica estática que constrói e devolve objetos, se a responsabilidade estiver na classe certa.
- Constante compartilhada e imutável que pertence ao conceito da classe.

Nunca compartilhe a mão sem intenção

Se `Mao` ou sua lista fosse static, todos os jogadores e todas as partidas observariam o mesmo estado. Um teste poderia contaminar o seguinte. Cada participante precisa de estado de instância próprio.

Os três sentidos que final pode ter

Uso	O que impede	O que não impede
Variável local final	Nova atribuição à variável.	Mutação do objeto referenciado.
Campo final	Trocar a referência depois da construção.	Alterar o estado interno do objeto.
Classe final	Criação de subclasses.	Mutação das instâncias.

Classifique os membros

- 1 Para cada método planejado, pergunte se ele precisa de uma instância específica.
- 2 Explique por que main pode ser static e uma mão não deveria ser.
- 3 Dê um exemplo de campo final que ainda aponta para objeto mutável.
- 4 Encontre um membro static atual cuja crítica é de responsabilidade, não de sintaxe.

Critério de conclusão: Você justifica static pelo modelo, não pela conveniência de chamada.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Membro de instância opera sobre um objeto escolhido. 2. static não recebe this. 3. Campo static compartilha estado entre instâncias. 4. final impede reatribuição, não garante imutabilidade profunda.	1. Qual baralho um método static sem parâmetro embaralharia? 2. Por que uma mão static é perigosa? 3. O que final List realmente promete?

CAPÍTULO 9

Responsabilidade única, coesão e acoplamento

Uma classe coesa reúne estado e comportamentos que explicam o mesmo conceito. O objetivo não é ter poucas linhas, mas ter um motivo principal e compreensível para mudar.

OBJETIVOS DE APRENDIZAGEM

- ❑ Usar estado e vocabulário do domínio para localizar métodos.
- ❑ Distinguir alta coesão de classe Deus.
- ❑ Reduzir conhecimento desnecessário entre objetos.

Três perguntas para localizar comportamento

- 1 Quem já possui os dados necessários para realizar a operação?
- 2 Que conceito do domínio esta operação descreve?
- 3 Qual classe teria motivo para mudar se essa regra fosse alterada?

Classe	Responsabilidade principal	Não deveria absorver
App	Interface textual e montagem inicial.	Pontuação, compra do dealer e manipulação de listas.
Blackjack	Orquestrar fases e regras da rodada.	Detalhes internos da coleção do baralho.
Baralho	Manter cartas restantes, embaralhar e entregar.	Decidir vencedor ou ler teclado.
Carta	Representar rank e naipe.	Fabricar um baralho ou decidir valor contextual do ás.
Mao	Guardar cartas e interpretar seu total.	Retirar cartas diretamente do baralho.
Jogador	Nome, mão e decisão do usuário.	Regra automática da banca.
Dealer	Mão e política automática de compra.	Menu do programa.

Coesão

Alta coesão significa que os elementos de uma classe trabalham para o mesmo propósito.

`Carta.criarNaipe()` é static de forma tecnicamente possível, mas semanticamente estranha: uma única carta passou a conhecer a criação de treze outras. A crítica é de coesão e vocabulário, não apenas de sintaxe.

Acoplamento

Acoplamento mede quanto uma parte precisa conhecer de outra. Blackjack precisa conhecer que Baralho pode entregar uma carta, mas não deveria saber se ele usa `ArrayList`, `Deque` ou outro armazenamento. Depender de comportamento público reduz o acoplamento com detalhes.

Evite a classe Deus

Blackjack é coordenador, mas isso não significa fazer tudo. Ele ordena a colaboração: pede uma carta ao baralho e a encaminha ao participante. Cada objeto continua responsável pela própria alteração de estado.

Jogo das responsabilidades

- 1 Escreva dez verbos do jogo: embaralhar, retirar, receber, calcular, decidir, comparar e outros.
- 2 Atribua cada verbo à classe que possui os dados necessários.
- 3 Para cada decisão, escreva uma alternativa e explique por que ela teria menor coesão.
- 4 Marque toda classe que começou a conhecer detalhes de mais de três outras classes.

Critério de conclusão: Cada método possui uma justificativa baseada em estado, domínio e motivo para mudar.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Responsabilidade única não significa um único método. 2. Coesão aproxima comportamento do estado relevante. 3. Acoplamento deve depender de contratos, não de detalhes internos. 4. Coordenar não significa executar todo o trabalho.	1. Quem conhece a lista que será embaralhada? 2. Por que <code>Carta.criarNaipes</code> tem baixa coesão? 3. Como Blackjack pode coordenar sem manipular listas?

CAPÍTULO 10

Composição, herança e injeção de dependência

Relações entre classes devem expressar significado. Reusar código é consequência; não é justificativa suficiente para dizer que uma coisa é outra.

OBJETIVOS DE APRENDIZAGEM

- Distinguir relações 'tem um' e 'é um'.
- Avaliar herança pela substituíbilidade, não apenas por atributos repetidos.
- Reconhecer App como possível raiz de composição da aplicação.

Composição: tem um

- Jogador **tem uma** mão.
- Dealer **tem uma** mão.
- Mão **tem cartas recebidas**.
- Blackjack **tem ou conhece** os participantes e o baralho durante a partida.

Herança: é um

Perguntar 'Dealer é um Jogador?' não possui resposta automática. Ambos têm mão, mas isso só prova estado semelhante. Uma possível abstração `Participante` pode surgir quando houver contrato comum real. Criá-la cedo demais pode esconder diferenças e aumentar complexidade.

Modelo	Benefício	Risco / pergunta
Dealer extends Jogador	Reuso imediato de nome/mão, se fizer sentido.	Dealer pode substituir qualquer Jogador sem quebrar expectativas?
Ambos extends Participante	Nomeia o papel comum e centraliza comportamento compartilhado.	Já existe comportamento comum suficiente ou é abstração prematura?
Classes separadas com Mao	Modelo simples e explícito por composição.	Pode repetir pequena quantidade de código.
Interface Participante	Define capacidade comum sem impor estado.	Há múltiplas implementações e necessidade de polimorfismo agora?

Teste da substituíbilidade

Se um método recebe Jogador, entregar um Dealer deveria preservar todas as expectativas desse método. Se isso parece estranho, a herança talvez esteja nomeando apenas reuso, não uma relação 'é um' verdadeira.

Raiz de composição

Algum lugar precisa criar os objetos do sistema e conectá-los. Em uma aplicação pequena, App pode cumprir esse papel: recebe dados externos, monta Baralho, Jogador, Dealer e Blackjack, e então chama uma operação pública. Isso é uma raiz de composição simplificada.

Dependência	Intrínseca ou externa?	Decisão a justificar
Mao de um participante	Frequentemente intrínseca.	O participante nasce sempre com mão vazia?
Baralho de uma partida	Pode ser fornecido.	Testes precisam controlar a ordem das cartas?
Scanner	Interface externa.	Deve entrar no domínio ou permanecer em App?
Fonte de aleatoriedade	Infraestrutura configurável.	Será necessário tornar testes determinísticos?

Debate arquitetural

- 1 Desenhe três alternativas para Jogador e Dealer compartilharem o conceito de mão.
- 2 Para cada alternativa, escreva uma frase 'é um' e uma frase 'tem um'.
- 3 Escolha quem monta os objetos no início e mantenha essa estratégia consistente.
- 4 Liste quais dependências você gostaria de controlar em um teste.

Critério de conclusão: Você consegue defender uma opção e também explicar seus custos, sem tratá-la como única solução correta.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Composição modela 'tem um'; herança modela 'é um'. 2. Herança deve respeitar substituíbilidade. 3. Repetição pequena não justifica hierarquia prematura. 4. Injeção torna dependências explícitas e testáveis.	1. Dealer é substituível por Jogador em todos os contextos? 2. Quem deve montar o grafo inicial de objetos? 3. Por que receber um baralho ajuda um teste?

CAPÍTULO 11

Enums, imutabilidade, identidade e igualdade

Carta é um excelente exemplo de objeto de valor: muda de lugar, mas seu rank e seu naipe não deveriam mudar depois da construção.

OBJETIVOS DE APRENDIZAGEM

- Substituir vocabulários abertos por tipos fechados quando apropriado.
- Distinguir final, coleção não modificável e objeto imutável.
- Separar identidade de referência e igualdade lógica.

Strings mágicas

Os textos "copas", "Copas" e "copass" são todos Strings válidas para o compilador. O domínio, porém, aceita apenas quatro naipes. Um enum transforma esse conjunto fechado em tipo.

EXEMPLO DE ENUM FORA DO JOGO

```
enum Sinal {  
    VERDE, AMARELO, VERMELHO  
}
```

Não use ordinal como regra

A posição de uma constante enum pode mudar quando o código é reorganizado. Pontuação e significado devem ser propriedades explícitas, não dependências acidentais de ordinal().

Imutabilidade da carta

Propriedade	Decisão típica para Carta	Benefício
Campos	Privados e final.	Identidade não muda após a construção.
Construtor	Recebe e valida rank e naipe.	Nenhuma carta parcialmente válida nasce.
Setters	Ausentes.	Carta não vira outra carta durante a rodada.
Consulta	Métodos sem efeito colateral.	Pode circular com segurança entre coleções.

Limite da palavra final

`private final Carta carta` impede trocar a referência do campo; não prova que Carta é imutável. A imutabilidade depende de toda a API e do estado alcançável pelo objeto.

==, equals e hashCode

Operação	Compara	Uso
<code>a == b</code>	Se as referências apontam para a mesma instância.	Identidade de referência.
<code>a.equals(b)</code>	Igualdade lógica definida pelo tipo.	Duas cartas com mesmo rank e naipe, se essa for a decisão do domínio.
<code>hashCode()</code>	Código consistente com equals.	HashSet, HashMap e testes de unicidade.
<code>toString()</code>	Representação textual legível.	Logs, terminal e diagnóstico.

O uso atual de `"J".equals(valorNominal)` é positivo: compara conteúdo e continua seguro se o argumento for null. Comparar conteúdo de String com `==` seria incorreto, pois `==` testa identidade da referência.

Objeto de valor

Um objeto de valor é definido principalmente por seus dados, não por uma identidade administrativa separada. Duas instâncias representando o mesmo rank e naipe podem ser consideradas logicamente iguais. Essa decisão facilita provar que o baralho possui as 52 combinações únicas.

Tabela de decisões da Carta

- 1 Liste todos os naipes e ranks válidos sem usar números mágicos.
- 2 Decida o que torna duas cartas logicamente iguais.
- 3 Explique por que uma dama continua sendo dama mesmo valendo dez pontos.
- 4 Liste as operações que poderiam existir sem alterar a carta.

Critério de conclusão: Você consegue explicar a diferença entre identidade, representação e pontuação contextual.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Enum modela vocabulário fechado. 2. Carta pode ser imutável e mover-se entre coleções. 3. <code>==</code> testa referência; <code>equals</code> pode testar valor lógico. 4. <code>hashCode</code> deve permanecer coerente com <code>equals</code>.	1. Por que String permite naipes inválidos? 2. O que torna uma Carta imutável? 3. Duas damas de copas distintas deveriam ser iguais neste domínio?

PARTE 3

Modelando o domínio Blackjack

Agora os conceitos se encontram no jogo: uma carta imutável, um baralho que controla cartas restantes, mãos independentes, participantes e um coordenador de estados.

NESTA PARTE

- Arquitetura e grafo de objetos
- Carta: rank, naipe e valor
- Baralho: geração, embaralhamento e retirada
- Mão: pontuação, ases e natural
- Participantes e fluxo da rodada
- Interface textual e Scanner

CAPÍTULO 12

O grafo de objetos desejado

POO aparece quando objetos com estado próprio colaboram por operações bem definidas. O desenho abaixo é uma arquitetura possível para a primeira versão, não uma verdade única.

OBJETIVOS DE APRENDIZAGEM

- ❑ Ler um sistema como um grafo de referências.
- ❑ Distinguir coordenação, propriedade e transferência.
- ❑ Entender por que uma mão começa vazia em vez de ser gerada aleatoriamente.

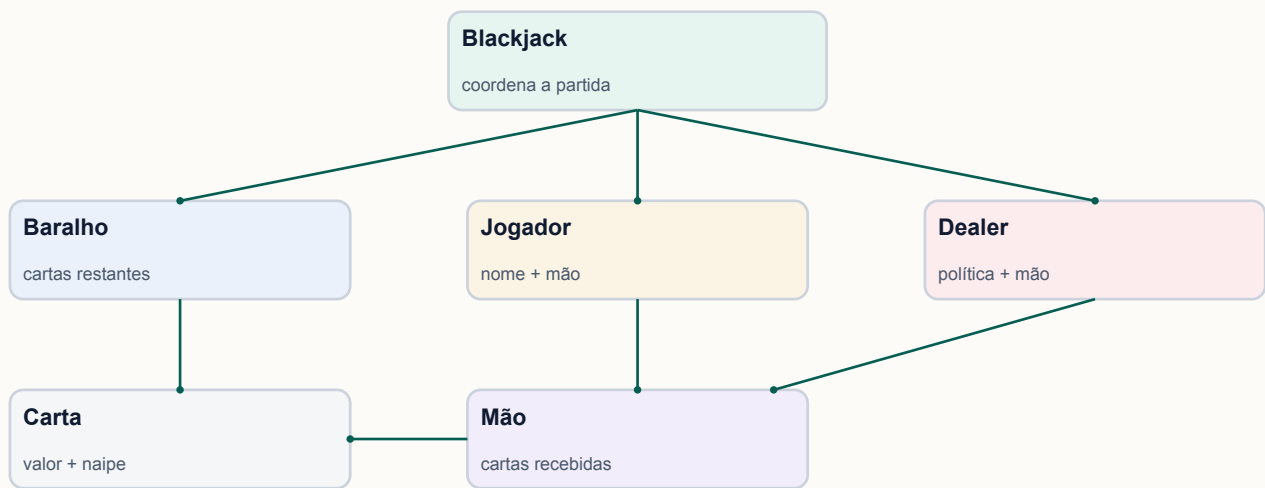


Figura 4. Grafo conceitual de objetos para uma rodada com um jogador e um dealer.

Como ler o desenho

- Blackjack conhece os participantes e o baralho porque coordena as fases.
- Baralho conhece suas cartas restantes por meio de uma coleção interna.
- Cada participante alcança sua própria Mão.
- Cada Mão alcança as cartas que recebeu.
- Carta não precisa conhecer Baralho, Mão, Jogador ou Dealer.

Mão não é sorteada isoladamente

A mão nasce vazia. Todas as cartas vêm de um único baralho compartilhado e são entregues uma por vez. Gerar mãos independentes poderia repetir a mesma carta e perder a conservação do conjunto.

Referências necessárias e desnecessárias

Relação	Por que existe	Relação inversa necessária?
Blackjack -> Baralho	O coordenador solicita cartas e preparação.	Baralho não precisa conhecer o jogo.
Jogador -> Mao	O jogador mantém seu estado de cartas.	Mao não precisa conhecer o dono para pontuar.
Mao -> Carta	A mão guarda o conjunto recebido.	Carta não precisa saber em qual mão está.
Blackjack -> participantes	O jogo alterna entregas e turnos.	Participantes podem ignorar detalhes do coordenador.

Estado distribuído

Nenhuma classe precisa copiar todos os dados. Blackjack não mantém outra lista de cartas da mão, nem Baralho mantém outra pontuação. Cada verdade possui uma fonte principal. Isso evita estados redundantes que poderiam discordar.

Fonte única de verdade

Se a pontuação pode ser derivada das cartas, guardar cartas e pontuação mutável ao mesmo tempo cria risco: uma nova carta pode entrar e o total permanecer antigo. Enquanto o custo é pequeno, recalcule o valor derivado.

Desenho do grafo real e desejado

- 1 Desenhe os objetos que existem logo após App criar a partida atual.
- 2 Inclua os objetos temporários criados pelos inicializadores de Blackjack.
- 3 Risque os que ficam inalcançáveis após o construtor.
- 4 Desenhe o grafo desejado e conte quantas mãos e baralhos permanecem.

Critério de conclusão: Seu desenho mostra referências, não apenas nomes de classes.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Objetos colaboram por referências e operações. 2. Cada informação deve ter uma fonte principal. 3. Mãos começam vazias e recebem cartas do mesmo baralho. 4. A relação inversa só existe quando há necessidade real de navegação.	1. Quem precisa conhecer Carta? 2. Por que Carta não precisa conhecer Mao? 3. Que estado não deveria ser duplicado em Blackjack?

CAPÍTULO 13

Projetando Carta: rank, naipe e pontuação

Uma carta tem identidade estável, mas sua contribuição para a pontuação pode depender do contexto. Misturar esses conceitos produz exatamente os três defeitos presentes no construtor atual.

OBJETIVOS DE APRENDIZAGEM

- Separar rank, representação e valor de pontuação.
- Detectar atribuições ausentes e sobrescritas.
- Definir uma carta válida e imutável.

Autópsia do construtor atual

CARTA.JAVA - TRECHO ATUAL

```
public Carta(String valorNominal, String naipe) {
    String c = "10";

    if ("J".equals(valorNominal)
        || "Q".equals(valorNominal)
        || "K".equals(valorNominal)) {
        this.valorNominal = c;
    }
    this.valorNominal = valorNominal;
}
```

Defeito	O que realmente acontece	Conceito
Naípe recebido, mas não atribuído	Todo campo naipe permanece null.	Construtor não estabelece invariante.
Gerador envia 1 a 13	O if que procura J, Q e K nunca é verdadeiro nesse fluxo.	Representações incompatíveis.
Atribuição depois do if	Mesmo uma entrada J teria o valor 10 imediatamente sobrescrito por J.	Fluxo é determinado por chaves, não indentação.

Quatro conceitos que não devem ser fundidos

Conceito	Exemplo para uma dama de copas
Rank / valor nominal	DAMA ou Q.
Naípe	COPAS.
Representação	Q de copas, possivelmente com símbolo na interface.
Valor-base no Blackjack	10.
Contribuição contextual	Para figuras é 10; para ás depende da mão.

Uma dama não vira dez

A carta continua sendo uma dama. Dez é a contribuição dela para uma regra de pontuação. Preservar o rank mantém apresentação, igualdade, testes e futuras regras consistentes.

Tabela de domínio

Ranks	Valor-base	Observação
2 a 10	O próprio número	Contribuição direta.
J, Q, K	10	O rank continua sendo figura.
A	1 ou 11	A Mao escolhe a interpretação que melhor atende ao conjunto.

Invariantes e operações de uma Carta

- Nasce com rank pertencente ao conjunto permitido.
- Nasce com naipe pertencente aos quatro valores permitidos.
- Nunca altera rank ou naipe durante sua vida.
- Pode informar representação e valor-base sem conhecer a mão.
- Pode definir igualdade lógica por rank e naipe, se essa for a decisão do projeto.

Matriz de decisão da Carta

- 1 Crie no papel treze linhas de rank e registre representação e valor-base.
- 2 Para cada campo, decida se pode mudar depois da construção.
- 3 Explique como o construtor prova que uma carta nunca nasce parcial.
- 4 Descreva três testes capazes de detectar os defeitos atuais de maneira independente.

Critério de conclusão: Sua modelagem preserva a identidade da carta e deixa o ás para o contexto da mão.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Rank e pontuação são conceitos diferentes. 2. O construtor deve armazenar e validar todos os dados obrigatórios. 3. Uma atribuição posterior pode sobrescrever uma anterior. 4. Carta é boa candidata a objeto de valor imutável.	1. Por que o naipe atual fica null? 2. Por que o if de figuras não é alcançado pelo gerador? 3. Quem decide se o ás vale 1 ou 11?

CAPÍTULO 14

Projetando Baralho: gerar, embaralhar e entregar

O baralho é o proprietário operacional das cartas restantes. Ele deve preservar o conjunto, alterar apenas a ordem ao embaralhar e transferir exatamente uma carta ao comprar.

OBJETIVOS DE APRENDIZAGEM

- Modelar as 52 combinações como produto de naipes e ranks.
- Especificar invariantes de embaralhamento e retirada.
- Tratar baralho vazio e aleatoriedade de maneira testável.

Produto cartesiano

Um baralho padrão sem curingas contém cada combinação de quatro naipes com treze ranks. Isso é um produto cartesiano: $4 \times 13 = 52$. Dois laços sobre conjuntos fechados expressam melhor essa regra que quatro chamadas repetidas com Strings.

Invariante de um baralho novo	Como verificar
Exatamente 52 cartas	Contar a coleção.
Exatamente 13 por naipe	Agrupar por naipe.
Exatamente 4 por rank	Agrupar por rank.
52 combinações únicas	Comparar tamanho da lista e de um conjunto lógico.
Nenhum campo nulo	Validar cada Carta.

Embaralhar é permutar

Embaralhar muda a ordem, não o conjunto. Um bom contrato não promete que a ordem final será sempre diferente: por probabilidade, uma permutação pode coincidir com a original. O teste robusto verifica que tamanho e multiconjunto foram preservados, ou usa uma fonte aleatória controlada.

Não invente um shuffle caseiro cedo

A biblioteca padrão já oferece embaralhamento adequado para este projeto. O aprendizado principal é colocar o comportamento no dono da coleção e testar invariantes, não reinventar o algoritmo.

Retirar é remover e devolver

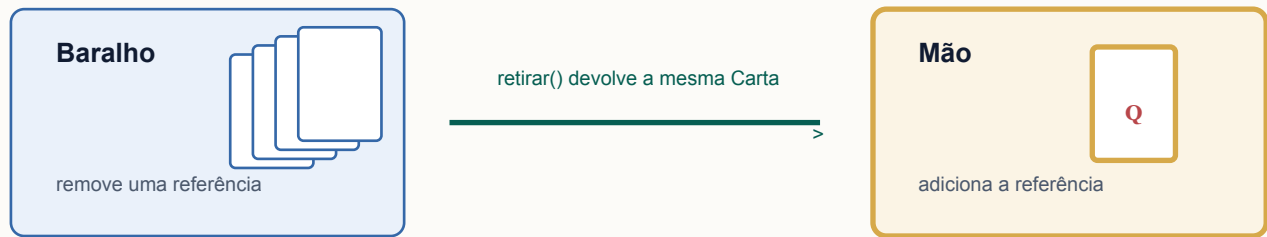


Figura 5. Transferência da mesma instância de Carta entre duas coleções.

A carta não é clonada nem recriada. A referência sai da coleção de cartas restantes e entra na coleção da mão. Uma invariante global útil é: cartas restantes + cartas em todas as mãos = cartas pertencentes à rodada.

Baralho vazio

Política	Vantagem	Custo
Lançar exceção clara	Falha cedo quando a operação viola o estado esperado.	Chamador precisa evitar ou tratar o caso.
Retornar Optional	Ausência fica explícita no tipo.	Adiciona conceito e tratamento ao fluxo.
Retornar null	Implementação simples.	Espalha risco de NullPointerException e esconde intenção.

Aleatoriedade testável

Para testar distribuição, é útil preparar um baralho em ordem conhecida ou fornecer uma fonte de aleatoriedade com semente. Isso é outra aplicação de injeção de dependência: a produção usa aleatoriedade real; o teste usa uma sequência previsível.

Especificação do Baralho

- 1 Escreva o contrato de criar, embaralhar, retirar e consultar quantidade.
- 2 Defina o que acontece ao retirar a última carta e tentar retirar novamente.
- 3 Liste testes separados para quantidade, distribuição por naipe e unicidade.
- 4 Escolha qual extremidade da lista representa o topo e mantenha essa decisão consistente.

Critério de conclusão: Seus contratos permitem provar conservação, unicidade e tratamento do vazio.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Baralho novo é o produto de quatro naipes e treze ranks. 2. Embaralhar preserva o conjunto. 3. Retirar remove e devolve a mesma instância. 4. Baralho vazio exige política explícita.	1. Por que testar somente tamanho 52 é insuficiente? 2. Por que um shuffle pode manter a mesma ordem? 3. Que quantidade global deve ser conservada ao distribuir?

CAPÍTULO 15

Projetando Mão: estado, pontuação e ases

A mão é mais que uma lista: ela concentra o contexto necessário para interpretar figuras, múltiplos ases, mão macia, estouro e blackjack natural.

OBJETIVOS DE APRENDIZAGEM

- Justificar Mao como classe com comportamento próprio.
- Calcular pontuação sem alterar as cartas.
- Distinguir total 21 de blackjack natural e mão macia de mão dura.

Por que Mao merece uma classe

Se ela fosse apenas um campo List dentro de Jogador, ainda seria possível começar. Mas as regras de pontuação dependem do conjunto completo e formam uma responsabilidade coesa. Mao pode garantir lista não nula, controlar recebimento, calcular total e responder perguntas de domínio sem expor a coleção.

Operação conceitual	Comando ou consulta	Invariante relacionada
Receber uma carta	Comando: muda estado.	Não aceita null e adiciona exatamente uma referência.
Quantidade de cartas	Consulta: não muda estado.	Resultado corresponde à coleção.
Calcular total	Consulta derivada.	Não altera Carta nem armazena valor desatualizado.
Verificar estouro	Consulta derivada.	Verdadeiro apenas quando total > 21.
Verificar natural	Consulta derivada.	Total 21 com exatamente duas cartas.

Algoritmo conceitual do ás

- 1 Some cartas numéricas pelo próprio valor.
- 2 Some J, Q e K como 10.
- 3 Conte inicialmente cada ás como 11 e registre quantos ases altos existem.
- 4 Enquanto o total ultrapassar 21 e ainda houver ás contado como 11, subtraia 10 e marque um ás como ajustado.
- 5 O total final é a melhor pontuação que não estoura, quando isso é possível.

PSEUDOCÓDIGO - NÃO É A IMPLEMENTAÇÃO JAVA

```
total = soma tratando cada ás como 11
asesAltos = quantidade de ases

enquanto total > 21 e asesAltos > 0:
    total = total - 10
    asesAltos = asesAltos - 1
```

Mão	Cálculo	Total	Classificação
A + 6	11 + 6	17	Macia: um ás ainda vale 11.
A + 9	11 + 9	20	Macia.
A + K	11 + 10	21	Blackjack natural.
A + A	22 - 10	12	Um ás vale 11, outro vale 1.
A + A + 9	31 - 10	21	Três cartas; não é natural.
A + 9 + 5	25 - 10	15	Mão dura após ajuste.
A + A + 9 + K	41 - 10 - 10	21	Quatro cartas; não é natural.
A + A + 10 + K	42 - 10 - 10	22	Estourou.
7 + 7 + 7	21	21	Total 21, não natural.
K + Q + 2	22	22	Estourou.

Mão macia e mão dura

Uma mão é macia quando pelo menos um ás ainda conta como 11. Ela é dura quando nenhum ás conta como 11. A mesma sequência pode mudar de macia para dura após receber outra carta, sem alterar o rank de nenhum ás.

Pontuação derivada

Guardar um campo total e também guardar cartas cria duas fontes de verdade. Se receber `Carta` esquecer de atualizar `total`, o objeto entra em contradição. Para coleções pequenas, recalcular é simples e seguro. Otimização só deve aparecer se uma medida real justificar.

Laboratório de ases

- 1 Calcule manualmente todos os casos da tabela sem olhar a coluna Total.
- 2 Crie dois exemplos que exigem ajustar dois ases.
- 3 Encontre um contraexemplo para a regra ingênua 'ás sempre vale 11'.
- 4 Explique por que A + K é natural e 7 + 7 + 7 não é.
- 5 Escreva testes antes do método de pontuação.

Critério de conclusão: Você calcula múltiplos ases e explica cada ajuste de 10, sem modificar as cartas.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Mão guarda cartas e interpreta o conjunto. 2. Ás começa em 11 e pode ser ajustado para 1. 3. Natural exige exatamente duas cartas. 4. Pontuação derivada evita estado redundante.	1. Por que <code>Carta</code> não decide sozinha o valor do ás? 2. Quando uma mão é macia? 3. Qual diferença existe entre 21 e blackjack natural?

CAPÍTULO 16

Participantes e fluxo completo da rodada

Jogador e dealer compartilham o fato de possuir uma mão, mas tomam decisões diferentes. Blackjack conecta essas decisões a uma sequência de estados válida.

OBJETIVOS DE APRENDIZAGEM

- Garantir estado independente para cada participante.
- Separar política de decisão e mecanismo de receber carta.
- Descrever uma rodada como transições ordenadas.

Uma mão por participante

Criar `Mao mao = new Mao()` dentro dos construtores atuais não estabelece a relação. Cada classe precisa manter uma referência de instância para sua própria mão. O campo não pode ser static, ou todos compartilhariam o mesmo estado.

Aspecto	Jogador	Dealer
Estado comum	Uma mão própria.	Uma mão própria.
Identificação	Nome fornecido pelo usuário.	Pode usar nome fixo ou papel 'banca'.
Decisão	Entrada externa: pedir ou parar.	Política automática.
Regra de compra	Escolhida pelo usuário enquanto permitido.	Compra até limite definido pela variante.

A rodada como máquina de estados



Figura 6. Estados conceituais da rodada e sua ordem principal.

A máquina de estados não precisa virar um enum imediatamente. Ela é primeiro uma ferramenta de raciocínio: certas operações só fazem sentido após outras. Não se compara vencedor antes de distribuir, nem se permite pedir carta depois de encerrar a rodada.

Distribuição inicial em rodadas

Passo	Destino	Cartas restantes
Estado inicial	Nenhum	52
Rodada 1, carta 1	Jogador	51
Rodada 1, carta 2	Dealer	50
Rodada 2, carta 1	Jogador	49
Rodada 2, carta 2	Dealer	48

Carta oculta é apresentação

A segunda carta do dealer pode estar presente em sua Mao e apenas ser ocultada na saída. Não é necessário manter uma mão incompleta ou uma carta fora do modelo para esconder informação do usuário.

Política do dealer

A regra de soft 17 varia entre mesas: algumas fazem o dealer parar, outras comprar. A primeira versão precisa declarar uma variante, por exemplo 'dealer para em qualquer 17'. Política é a regra de decisão; retirar e receber carta continuam sendo mecanismos dos objetos responsáveis.

Determinação do resultado

Condição avaliada	Resultado típico
Jogador estourou	Dealer vence, independentemente do total posterior.
Dealer estourou e jogador não	Jogador vence.
Natural versus 21 comum	Natural possui tratamento superior nas regras usuais adotadas.
Totais válidos diferentes	Maior total vence.
Totais válidos iguais	Empate, também chamado push.

Ordem das condições importa

Uma cadeia mal ordenada pode declarar vencedor alguém que estourou ou tratar natural como 21 comum. Escreva uma tabela de decisão antes do if/else e teste uma linha de cada vez.

Simulação em papel

- 1 Use um baralho numerado e simule as quatro entregas iniciais, registrando contagens.
- 2 Marque em qual estado cada operação pode ocorrer.
- 3 Escolha e documente a política de soft 17.
- 4 Monte casos para todas as linhas da tabela de resultado.

Critério de conclusão: Você consegue narrar uma rodada completa sem acessar diretamente nenhuma lista interna.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Cada participante possui mão independente. 2. Blackjack coordena transições e não duplica estado. 3. Distribuição inicial entrega quatro cartas no cenário básico. 4. Política do dealer precisa declarar a variante adotada.	1. Quantas cartas restam após a distribuição inicial? 2. Por que ocultar uma carta não muda o modelo da mão? 3. Que condições devem ser avaliadas antes de comparar totais?

CAPÍTULO 17

Interface textual, Scanner e controle de fluxo

Entrada e saída são fronteiras externas. Mantê-las separadas do domínio permite testar o Blackjack sem digitar no terminal e evita que Scanner contamine as regras.

OBJETIVOS DE APRENDIZAGEM

- Prever o conteúdo restante no buffer do Scanner.
- Manter menu e regras do jogo em camadas distintas.
- Projetar fluxo coerente para saída, opções inválidas e EOF.

Tokens versus linhas

MODELO DO BUFFER

```
entrada: "1\nOtavio\n"

nextInt() -> consome "1"
nextLine() -> consome o resto da linha: ""
```

`nextInt()` lê o próximo token numérico, mas não consome automaticamente toda a linha. O `nextLine()` seguinte encontra primeiro a quebra de linha restante e devolve texto vazio.

Estratégia	Como funciona	Cuidado
Leitura por tokens	Usa <code>nextInt</code> , <code>next</code> e semelhantes.	Precisa administrar separadores e token inválido.
Leitura por linhas	Usa <code>nextLine</code> e converte o texto.	Precisa tratar falha de conversão explicitamente.

Coerência do menu atual

- O laço termina com opção 0, mas o menu não mostra 0.
- As opções 2 e 3 aparecem, porém não possuem comportamento.
- Não existe caso default para opção inválida.
- O switch tradicional ainda não tem break; novos cases abaixo causariam fall-through.
- Entrada não numérica lança exceção; uma repetição mal feita pode não consumir o token inválido.

Criar não é iniciar

O método `playBlackjack` atual cria os objetos e termina. O nome promete jogar, mas nenhum método embaralha, distribui ou inicia a rodada. Construção estabelece estado; uma operação explícita conduz o fluxo.

Separação de camadas

App pode perguntar nome, mostrar opções e converter entrada. Blackjack recebe decisões já interpretadas e aplica regras. Carta, Mao e Baralho não deveriam conhecer Scanner nem System.out.

Robustez da fronteira

Entrada problemática	Decisão da interface
Nome nulo ou em branco	Pedir novamente antes de criar jogador válido.
Texto onde se espera número	Explicar formato e repetir sem travar o buffer.
Opção inexistente	Mostrar mensagem e retornar ao menu.
Fim de entrada (EOF)	Encerrar de forma previsível em execução automatizada.
Opção ainda não implementada	Não anunciar ou informar claramente a indisponibilidade.

Rastreamento de entrada

- 1 Escreva exatamente quais caracteres estão no buffer antes e depois de cada leitura.
- 2 Desenhe um menu em que todas as opções exibidas tenham tratamento coerente.
- 3 Decida uma estratégia única de leitura e explique como ela trata texto inválido.
- 4 Renomeie mentalmente `playBlackjack` conforme o comportamento que ele realmente possui hoje.

Critério de conclusão: Seu fluxo aceita nome válido, sai com opção visível e não coloca regras de pontuação em App.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. <code>nextInt</code> e <code>nextLine</code> consomem unidades diferentes. 2. Menu e condição de saída devem concordar. 3. Domínio não deve depender de Scanner ou System.out. 4. Construir um jogo não executa automaticamente uma rodada.	1. Por que o nome atual tende a ficar vazio? 2. Que opções do menu não têm tratamento? 3. Qual classe deve transformar entrada textual em decisão?

PARTE 4

UML e PlantUML com significado

Um diagrama útil não é decoração nem cópia automática do código. Ele mostra decisões, direção de conhecimento, multiplicidades e fases do comportamento em um nível adequado ao problema.

NESTA PARTE

- Sintaxe essencial do PlantUML
- Associação, dependência, composição e herança
- Auditoria do `blackjack.puml` atual
- Diagramas de sequência e estado

CAPÍTULO 18

PlantUML: da sintaxe ao modelo

O arquivo .puml é texto declarativo que uma ferramenta renderiza. A sintaxe permite desenhar rapidamente, mas a qualidade depende das decisões de modelagem que você escreve.

OBJETIVOS DE APRENDIZAGEM

- Estruturar um documento PlantUML válido.
- Representar campos e operações com tipos e visibilidade.
- Escolher o nível de detalhe conforme o objetivo do diagrama.

Envelope mínimo

ESTRUTURA GENÉRICA

```
@startuml
title Nome do diagrama

class Exemplo {
    - estado: Tipo
    + executar(entrada: Tipo): Retorno
}

@enduml
```

O arquivo atual começa diretamente com `title Blackjack` e não possui os marcadores de início e fim. Algumas integrações podem tentar interpretar fragmentos, mas o documento PlantUML completo deve declarar `@startuml` e `@enduml`.

Campos e operações

Elemento	Forma UML	Pergunta que responde
Campo	- mao: Mao	Que estado a instância guarda?
Operação	+ receber(c: Carta): void	Que comportamento a API oferece?
Construtor	+ Jogador(nome: String)	Como a instância nasce válida?
Membro static	{static} ou sublinhado	O membro pertence à classe?
Tipo abstrato	abstract class	A classe representa conceito incompleto?

Sintaxe não substitui semântica

A linha '+ copas.criarNaipes' pode até aparecer como texto em um compartimento, mas não representa corretamente campo, operação nem chamada UML. Copas é valor do domínio; criar cartas é comportamento. Primeiro classifique o conceito, depois escolha a notação.

Quanto detalhe colocar

Objetivo do diagrama	Detalhe recomendado
Explorar domínio	Classes principais, responsabilidades e relações; poucos métodos.
Planejar implementação	Campos relevantes, assinaturas públicas e multiplicidades.
Documentar API	Operações públicas precisas, tipos e restrições importantes.
Explicar comportamento	Diagrama de sequência ou estados, não uma classe gigante.

Três diagramas para três perguntas

Diagrama	Pergunta central	Aplicação no projeto
Classes	Que tipos existem e como se relacionam?	Estrutura de Carta, Mao, Baralho e participantes.
Sequência	Quem chama quem e em qual ordem?	Distribuição de uma carta e turnos.
Estados	Que fases existem e quais transições são válidas?	CRIADA, DISTRIBUÍDA, TURNO e ENCERRADA.

Renderização incremental

- 1 Adicione apenas `@startuml` e `@enduml` e renderize.
- 2 Corrija uma classe por vez usando nome: Tipo e metodo(): Retorno.
- 3 Remova do diagrama qualquer linha que você não consegue classificar como campo ou operação.
- 4 Defina no título ou em uma nota se o desenho representa o código atual ou o projeto desejado.

Critério de conclusão: O arquivo renderiza e cada membro possui significado UML explicável.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Documento PlantUML possui marcadores de início e fim. 2. Campo usa nome e tipo; método usa parâmetros e retorno. 3. O nível de detalhe depende da pergunta. 4. Comportamento temporal pede sequência ou estados.	1. Por que '- Mao' é incompleto? 2. Como representar um método público que devolve Carta? 3. Que diagrama explica melhor a distribuição inicial?

CAPÍTULO 19

Relações UML, direção e multiplicidade

Setas e diamantes expressam compromissos semânticos. Usá-los apenas porque 'parecem relação' pode produzir um desenho mais enganoso que a ausência de desenho.

OBJETIVOS DE APRENDIZAGEM

- ☐ Ler navegabilidade a partir de quem guarda ou usa a referência.
- ☐ Escolher associação, dependência, agregação, composição ou herança.
- ☐ Interpretar multiplicidades nos dois extremos da relação.

PlantUML	Nome	Significado conceitual
A ..> B	Dependência	A usa B temporariamente, por parâmetro, variável local ou chamada.
A --> B	Associação navegável	A mantém conhecimento duradouro de B.
A o-- B	Agregação	Todo-parte fraca; semântica frequentemente ambígua.
A *-- B	Composição	A é todo e B é parte exclusiva com ciclo de vida fortemente ligado.
A -- > B	Generalização	A é subtipo de B e deve ser substituível por B.

A direção vem do conhecimento

Se Jogador possui um campo do tipo Mao, Jogador conhece Mao. A navegação parte de Jogador. Isso não exige que Mao possua referência inversa para Jogador. Relações bidirecionais aumentam acoplamento e só devem existir quando ambos os sentidos são necessários.

Diamante fica no todo

Em uma composição Jogador *-- Mao, o diamante fica junto de Jogador, que representa o todo. O símbolo não indica direção de chamada; representa propriedade e ciclo de vida.

Como ler multiplicidade

Notação	Leitura
1	Exatamente uma instância.
0..1	Nenhuma ou uma.
0..*	Nenhuma ou muitas.
1..*	Uma ou muitas.
0..52	Entre zero e cinquenta e duas.

A multiplicidade escrita perto de uma classe indica quantas instâncias daquela classe podem se relacionar com uma instância do lado oposto. Leia a frase nos dois sentidos; muitas contradições aparecem somente na leitura inversa.

O caso especial das cartas que se movem

A mesma Carta passa de 'restante no baralho' para 'recebida por uma mão'. Usar simultaneamente dois diamantes pode sugerir propriedade permanente ou compartilhada. Associações simples, com nomes de papéis e uma nota de exclusividade, podem ser mais honestas que agregações decorativas.

Ressalva ao diagrama atual

As duas composições 1 para 1 de Jogador e Dealer com Mao são fortes demais em UML formal: lidas literalmente, podem sugerir que toda Mao participa obrigatoriamente das duas associações. Uma futura abstração Participante permitiria uma única composição; outra opção é usar associações opcionais e documentar exclusividade.

Split muda a multiplicidade

Uma mão por jogador é coerente na versão sem split. Ao adicionar split, um participante pode controlar várias mãos durante a rodada. Isso mostra que multiplicidade não é enfeite: ela documenta o escopo funcional adotado.

Leia cada relação como frase

- 1 Para cada linha do seu PUM, leia do lado esquerdo para o direito.
- 2 Leia novamente do lado direito para o esquerdo usando as multiplicidades.
- 3 Marque relações que indicam conhecimento duradouro e relações que são apenas uso temporário.
- 4 Explique o ciclo de vida prometido por cada diamante; remova o símbolo se não houver promessa clara.

Critério de conclusão: Nenhuma seta ou multiplicidade permanece apenas porque ficou visualmente bonita.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Navegabilidade segue quem conhece a referência. 2. Composição promete propriedade forte e exclusividade. 3. Multiplicidade precisa ser lida nos dois sentidos. 4. Agregação não é resposta automática para 'tem um'.	1. Onde fica o diamante da composição? 2. O que 0..52 próximo de Carta significa? 3. Por que Carta em trânsito exige cuidado com diamantes?

CAPÍTULO 20

Auditoria do blackjack.puml e evolução do modelo

Seu arquivo já identifica as classes e a intenção geral das relações. Agora ele precisa declarar se representa o código de hoje ou o desenho que você deseja alcançar.

OBJETIVOS DE APRENDIZAGEM

- ☐ Encontrar contradições factuais entre PUMML e Java.
- ☐ Separar modelo atual de modelo-alvo.
- ☐ Atualizar o diagrama em pequenos passos verificáveis.

Inventário de divergências

PUMML atual	Código atual	Decisão necessária
Sem @startuml / @enduml	Arquivo é apenas fragmento.	Completar o envelope PlantUML.
Blackjack contém playBlackjack	O método está em App.	Mover comportamento ou atualizar diagrama.
Baralho contém embaralhar privado	Método está público, static e vazio em Blackjack.	Escolher responsabilidade e sincronizar.
Carta contém criarNaipes privado	Método é público e static.	Corrigir notação e discutir coesão.
Jogador e Dealer possuem Mao	Construtores criam local, mas campos não existem.	Implementar relação ou marcar desenho como alvo.
copas.criarNaipes e similares	São chamadas feitas no construtor de Baralho.	Não listar chamadas como membros de classe.
gerarBaralho()	Não existe como método; ocorre no construtor.	Modelar o código real ou planejar refatoração.

Atual versus desejado

Estratégia	Vantagem	Como rotular
Diagrama do estado atual	Ajuda a depurar e documentar o que existe.	Título: 'Modelo implementado - versão X'.
Diagrama de projeto	Ajuda a raciocinar antes de codificar.	Título: 'Modelo proposto - versão 1'.
Dois diagramas	Torna a evolução e as diferenças explícitas.	Use nomes de arquivo ou notas distintas.

Evite o diagrama híbrido

Misturar métodos reais, relações desejadas e operações imaginadas sem rótulo cria contradições. Um leitor não sabe se deve confiar no desenho como documentação ou como plano.

Diagrama de sequência da distribuição

EXEMPLO CONCEITUAL DE SEQUÊNCIA

```
@startuml
Blackjack -> Baralho : retirarCarta()
Baralho --> Blackjack : carta
Blackjack -> Jogador : receberCarta(carta)
Jogador -> Mao : adicionarCarta(carta)
@enduml
```

Esse desenho responde 'quem chama quem e em qual ordem?'. Ele não substitui o diagrama de classes, que responde 'quem conhece quem de maneira estrutural?'. Observe também que o coordenador pode entregar ao Jogador, que delega à própria Mao; falar diretamente com Mao seria outra decisão com trade-offs.

Ciclo de manutenção



Figura 7. O PUML participa do ciclo de projeto, não é produzido apenas no fim.

Plano de correção do diagrama

- 1 Escolha se o arquivo será atual ou proposto e escreva isso no título.
- 2 Corrija envelope, campos e operações sem alterar ainda as relações.
- 3 Compare cada método com a classe e a visibilidade no Java.
- 4 Revise relações e multiplicidades lendo-as nos dois sentidos.
- 5 Crie separadamente uma sequência para a distribuição inicial.

Critério de conclusão: Uma pessoa consegue distinguir estrutura atual, intenção futura e fluxo temporal.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. PUML e Java não devem se contradizer sem rótulo. 2. Modelo atual e modelo-alvo cumprem papéis diferentes. 3. Sequência explica chamadas; classes explicam estrutura. 4. Diagramas evoluem junto com pequenos passos de código.	1. Onde playBlackjack existe hoje? 2. Que relação está desenhada, mas não implementada? 3. Qual diagrama mostra melhor a carta sendo entregue?

PARTE 5

Testes, diagnóstico e refatoração

Testes transformam regras em experimentos repetíveis. Depuração separa sintoma de causa; refatoração reorganiza com proteção para não alterar comportamento sem perceber.

NESTA PARTE

- Arrange, Act, Assert
- Matriz de invariantes e casos de borda
- Aleatoriedade determinística
- Compilador, stack trace e hipótese
- Refatoração segura

CAPÍTULO 21

O que torna um teste útil

Um teste é um experimento: prepara condições, executa uma ação e mede um efeito observável. Se ele não puder falhar quando a regra quebra, não está especificando comportamento.

OBJETIVOS DE APRENDIZAGEM

- Estruturar testes com Arrange, Act, Assert.
- Dar nomes que descrevem cenário e resultado.
- Diferenciar teste de unidade, integração e fumaça.



Figura 8. Estrutura mínima de um experimento automatizado.

O teste atual

APPTTEST.JAVA - TESTE ATUAL

```
@Test
void testApp() {
    assertTrue(true);
}
```

Esse teste confirma apenas que o JUnit consegue executar um método anotado. A condição é constante; nenhum defeito no Blackjack pode torná-la falsa. Ele é útil como verificação inicial do ambiente, mas não como teste do domínio.

Teste precisa ter poder de detecção

Pergunte: que alteração incorreta faria este teste falhar? Se a resposta for 'nenhuma', ele não protege comportamento. Um teste bom não precisa ser complexo; precisa observar uma regra relevante.

Arrange, Act, Assert

Fase	Pergunta	Exemplo conceitual
Arrange	Que cenário preciso preparar?	Criar uma mão vazia e uma carta válida.
Act	Qual única ação principal será executada?	Mandar a mão receber a carta.
Assert	Que efeito observável prova a regra?	Quantidade passou de zero para um.

ESQUELETO DELIBERADAMENTE INCOMPLETO

```
@Test
void operacaoAlteraOEstadoEsperado() {
    // Arrange: prepare dados claros
    // Act: execute uma única ação central
    // Assert: verifique o efeito observável
}
```

Nomes que documentam comportamento

Nome fraco	Nome descritivo
testMao	maoNovaComecaSemCartas
testCarta	cartaConservaRankENaipeRecebidos
testBaralho	retirarCartaReduzQuantidadeEmUm
testBlackjack	distribuicaoInicialEntregaDuasCartasParaCadaParticipante

Unidade versus integração

Nível	Escopo	Exemplo
Unidade	Uma classe e seu contrato público.	Pontuação de Mao preparada com cartas conhecidas.
Integração	Colaboração entre objetos.	Carta sai do Baralho e entra na Mao.
Fluxo	Várias etapas da aplicação.	Distribuição inicial com baralho controlado.
Interface	Entrada e saída textual.	Menu recebe opção e mostra resposta; use apenas onde necessário.

Transforme regra em experimento

- 1 Escolha uma invariante simples, como 'Mao começa vazia'.
- 2 Escreva Arrange, Act e Assert em português.
- 3 Diga qual defeito faria o teste falhar.
- 4 Dê ao teste um nome que inclua cenário e resultado.

Critério de conclusão: O teste possui ao menos uma observação que seria diferente se a regra quebrasse.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Teste útil pode falhar por uma quebra relevante. 2. AAA separa cenário, ação e observação. 3. Nome do teste documenta regra e contexto. 4. Comece por unidades antes de fluxos aleatórios completos.	1. O que <code>assertTrue(true)</code> prova? 2. Qual é a ação central no teste de receber carta? 3. Quando um teste de integração é necessário?

CAPÍTULO 22

Matriz de testes e casos de borda

Um único teste de tamanho não prova um baralho correto. Cada invariante composta precisa de observações independentes e casos de borda escolhidos para quebrar algoritmos ingênuos.

OBJETIVOS DE APRENDIZAGEM

- ❑ Transformar invariantes em uma matriz de testes.
- ❑ Escolher casos normais, limites e falhas.
- ❑ Distinguir identidade com `assertSame` de igualdade com `assertEquals`.

Carta

Cenário	O que observar	Bug que detecta
Carta válida	Rank e naipe recebidos são conservados.	Naipe nunca atribuído.
Figura	Rank continua J/Q/K e valor-base é 10.	Mistura rank e pontuação.
Entrada inválida	Construção é rejeitada conforme contrato.	Objeto parcialmente válido.
Duas cartas equivalentes	<code>equals/hashCode</code> seguem decisão do domínio.	Unicidade impossível de verificar logicamente.
Representação	<code>toString</code> é legível.	Saída padrão <code>Classe@hash</code> .

Baralho

Cenário	Observação principal
Novo	52 cartas, 13 por naipe, 4 por rank, 52 combinações únicas.
Conteúdo	Nenhum rank ou naipe nulo.
Uma retirada	Tamanho reduz em um e carta devolvida pertencia ao conjunto anterior.
Até esvaziar	Nenhuma carta se repete e exatamente 52 retiradas válidas ocorrem.
Vazio	Política de erro ou ausência é cumprida.
Embaralhar	Tamanho e conjunto são preservados.

Tamanho 52 não basta

O código atual já cria 52 objetos, mas todos podem ter naipe null. Uma propriedade composta exige testar quantidade, distribuição, validade e unicidade separadamente.

Mão

Grupo	Casos mínimos
Estado	Começa vazia; receber aumenta; limpar restaura vazio.
Independência	Duas mãos não são a mesma instância; mudar uma não altera a outra.
Números e figuras	2 + 7; K + Q; K + Q + 2.
Ás único	A + 6; A + 9 + 5; A + K.
Múltiplos ases	A + A; A + A + 9; A + A + 10 + K.
Natural	A + K é natural; 7 + 7 + 7 não é.
Entrada inválida	Receber null é rejeitado.

Identidade e igualdade nos asserts

Assert conceitual	Pergunta
assertSame	É exatamente a mesma instância? Útil para provar transferência da carta.
assertNotSame	São instâncias diferentes? Útil para independência das mãos.
assertEquals	Os valores são logicamente iguais? Depende de equals.
assertThrows	A operação rejeita corretamente o estado ou argumento inválido?

Integração da distribuição

- Jogador termina com duas cartas.
- Dealer termina com duas cartas.
- Baralho termina com 48 cartas no cenário de um jogador e um dealer.
- As quatro cartas entregues não permanecem entre as cartas restantes.
- A ordem das entregas segue as duas rodadas definidas.

Crie sua matriz antes dos testes

- 1 Escolha uma classe e liste estado normal, limite inferior, limite superior e entrada inválida.
- 2 Para cada linha, escreva qual assert expressaria a observação.
- 3 Marque testes que dependem de equals e testes que dependem de identidade.
- 4 Procure um algoritmo ingênuo que passaria nos casos normais e falharia na borda.

Critério de conclusão: Sua matriz detecta mais de um tipo de defeito por classe e inclui casos negativos.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Invariantes compostas pedem múltiplas verificações. 2. Bordas são escolhidas para quebrar simplificações. 3. Identidade e igualdade respondem perguntas diferentes. 4. Testes de integração provam colaboração e conservação.	1. Que teste detecta naipe null? 2. Como provar que a mesma Carta foi transferida? 3. Que caso exige ajustar dois ases?

CAPÍTULO 23

Aleatoriedade, depuração e hipóteses

Depurar não é alterar linhas ao acaso. É reproduzir um sintoma, formular uma hipótese sobre a causa e criar uma observação capaz de refutá-la.

OBJETIVOS DE APRENDIZAGEM

- Testar propriedades aleatórias sem exigir resultados impossíveis.
- Ler mensagens do compilador e stack traces por camadas.
- Aplicar um ciclo sistemático de hipótese e evidência.

O erro ao testar shuffle

É matematicamente possível embaralhar e obter a mesma ordem. Portanto, um teste que exige 'a ordem sempre muda' pode falhar mesmo com implementação correta. Teste propriedades determinísticas: quantidade, conjunto e validade permanecem. Para ordem específica, controle a fonte de aleatoriedade.

Teste frágil	Teste robusto
A ordem depois de shuffle deve ser diferente.	O conjunto e o tamanho são preservados.
Distribuição usa baralho aleatório real.	Distribuição usa baralho preparado para prever cartas.
Rodar cem vezes e esperar 'parecer aleatório'.	Testar contrato e delegar algoritmo à biblioteca padrão.

Ciclo de depuração

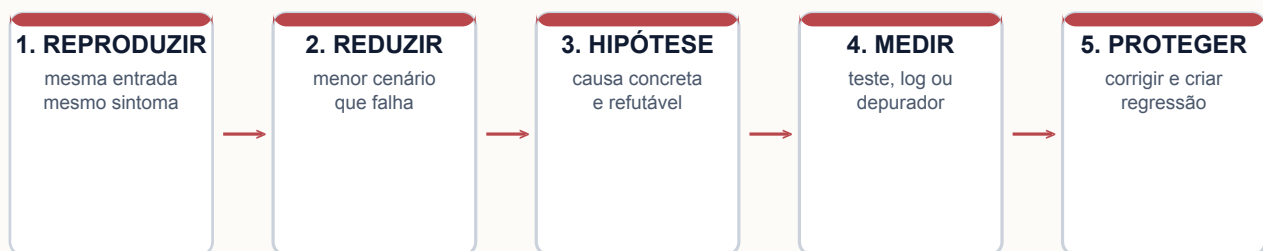


Figura 9. Ciclo científico de depuração.

Como ler um erro de compilação

- 1 Comece pelo primeiro erro, não pelas dezenas de consequências.
- 2 Localize arquivo e linha.
- 3 Identifique tipo esperado e tipo encontrado.
- 4 Leia a assinatura do método ou construtor chamado.
- 5 Pergunte qual decisão do modelo produziu o valor incorreto.

Como ler um stack trace

A primeira linha informa o tipo da exceção e sua mensagem. Em seguida, procure o primeiro quadro pertencente ao seu pacote. Ele mostra onde o erro foi observado, não necessariamente onde a causa nasceu. Um null pode ser usado em Mao, mas ter sido criado por Carta muitos passos antes.

Faça uma mudança por hipótese

Alterar Carta, Baralho, Mao e App simultaneamente impede saber qual mudança resolveu ou criou o problema. Reduza, observe, faça uma alteração pequena, compile e teste novamente.

Ferramentas de observação

Ferramenta	Quando ajuda	Cuidado
Mensagem do compilador	Tipos, símbolos, sintaxe e assinaturas.	Corrija o primeiro erro causal.
Teste unitário	Reprodução pequena e permanente.	Não dependa de ordem aleatória.
Breakpoint	Inspecionar campos e sequência em execução.	Observe antes e depois da linha suspeita.
Impressão temporária	Confirmar fluxo rapidamente.	Remova ruído e não substitua testes.
PUML / grafo	Visualizar referência ausente ou compartilhada.	Mantenha sincronizado com o código real.

Hipótese sobre o nome vazio

- 1 Escreva a entrada exata, incluindo quebras de linha.
- 2 Preveja o que cada chamada do Scanner consome.
- 3 Formule a hipótese em uma frase refutável.
- 4 Escolha uma observação mínima que confirme ou refute a hipótese.
- 5 Depois da correção, descreva um teste de regressão possível.

Critério de conclusão: Você identifica causa no buffer, não apenas adiciona chamadas até 'funcionar'.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Aleatoriedade deve ser testada por invariantes ou controlada. 2. Depuração parte de hipótese refutável. 3. Stack trace mostra onde o sintoma foi observado. 4. Bug corrigido merece teste de regressão.	1. Por que shuffle pode manter a ordem? 2. Qual é o primeiro passo da depuração? 3. Como reduzir o caso do nome vazio?

CAPÍTULO 24

Refatoração segura e evolução incremental

Refatorar muda a estrutura preservando comportamento observável. Quando o comportamento ainda está quebrado, primeiro corrija e proteja a regra; depois reorganize.

OBJETIVOS DE APRENDIZAGEM

- ☐ Distinguir correção de bug, nova funcionalidade e refatoração.
- ☐ Mover responsabilidades com proteção de testes.
- ☐ Evitar abstrações prematuras e mudanças grandes demais.

Tipo de mudança	Objetivo	Exemplo
Correção	Alterar comportamento incorreto.	Conservar o naipe recebido pela Carta.
Funcionalidade	Adicionar comportamento observável.	Distribuir cartas iniciais.
Refatoração	Melhorar estrutura sem mudar resultado.	Mover embaralhar para o dono do estado.

Sequência segura

- 1 Escolha uma regra pequena e escreva um teste que falha.
- 2 Faça a menor mudança para a regra passar.
- 3 Execute toda a suíte.
- 4 Somente então renomeie, mova ou extraia comportamento.
- 5 Execute novamente e atualize o PUMML correspondente.

Refatorações úteis neste projeto

Sinal	Refatoração possível	Pré-requisito
Método na classe errada	Mover método para o dono do estado.	Teste do comportamento atual ou desejado.
Campo criado e substituído	Remover inicialização redundante.	Construtor garante dependência válida.
Strings repetidas	Introduzir enum.	Testes de rank/naipe e decisão de domínio.
Trecho longo de condição	Extrair método com nome de intenção.	Casos de decisão protegidos.
Duplicação real em participantes	Avaliar Participante comum.	Contrato compartilhado e substituíbilidade claros.

Não refatore no escuro

Sem observações de comportamento, você não sabe se a reorganização preservou a regra. Testes pequenos atuam como cinto de segurança, não como prova absoluta de ausência de defeitos.

Abstração no momento certo

Participante, estratégia de dealer, interface de baralho e fonte aleatória são abstrações possíveis. Elas não precisam entrar juntas. Introduza uma abstração quando existir variação, duplicação significativa ou necessidade de teste que ela resolva de forma clara.

Plano de uma refatoração

- 1 Escolha um método que parece pertencer à classe errada.
- 2 Escreva o comportamento que deve permanecer igual.
- 3 Defina o teste de proteção.
- 4 Liste passos mecânicos pequenos para mover o comportamento.
- 5 Defina a atualização necessária no PUMML.

Critério de conclusão: Cada passo compila, os testes continuam passando e a intenção final fica mais clara.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Correção, funcionalidade e refatoração têm objetivos diferentes. 2. Refatoração deve preservar comportamento observado. 3. Mudanças pequenas permitem localizar causa e efeito. 4. Abstrações entram quando resolvem uma pressão real.	1. Mover embaralhar é correção ou refatoração no estado atual? 2. Que teste protege uma transferência de método? 3. Quando Participante deixa de ser prematuro?

PARTE 6

Roteiro de prática deliberada

Conhecimento se consolida quando cada conceito vira uma previsão, uma mudança pequena e uma prova objetiva. Esta parte organiza o projeto em marcos que você consegue concluir sem copiar uma solução inteira.

NESTA PARTE

- Dez marcos incrementais
- Exercícios em cinco níveis
- Checklists de revisão
- Autoavaliação e extensões

CAPÍTULO 25

Dez marcos verificáveis

Cada marco deve deixar o projeto compilando e acrescentar uma única capacidade observável. Não avance apenas porque escreveu código; avance quando consegue provar o critério.

OBJETIVOS DE APRENDIZAGEM

- ☐ Dividir a construção em incrementos de baixo risco.
- ☐ Definir critério de pronto antes de implementar.
- ☐ Manter testes e diagrama sincronizados com cada marco.

Marco 0 - base confiável

Meta	Crítérios de conclusão
Projeto compila	mvn test termina com sucesso e nenhuma assinatura está incompatível.
Teste pode falhar	Substituir a tautologia por uma primeira regra observável.
Escopo declarado	Versão inicial: um jogador, um dealer, um baralho, sem apostas e sem split.

Marcos 1 a 3 - estado válido

Marco	Entrega	Provas mínimas
1. Estado persistente	Mao guarda coleção; cada participante guarda sua Mao.	Mão começa vazia; duas mãos são independentes.
2. Carta válida	Rank e naipe conservados; representação coerente.	Nenhum campo nulo; figuras mantêm identidade.
3. Baralho válido	52 combinações únicas.	52 total, 13 por naipe, 4 por rank, sem duplicatas.

Marcos 4 a 6 - colaboração e regras

Marco	Entrega	Provas mínimas
4. Transferência	Uma Carta sai do Baralho e entra na Mao.	Mesma instância, tamanhos alterados, total conservado.
5. Distribuição	Duas rodadas alternadas.	Duas cartas para cada participante e 48 restantes.
6. Pontuação	Números, figuras, ases e natural.	Matriz completa de casos e múltiplos ases.

Marcos 7 a 9 - aplicação completa

Marco	Entrega	Provas mínimas
7. Rodada	Turnos, dealer e resultado.	Tabela de decisão coberta; nenhum estado impossível.
8. Interface	Menu e entrada robustos.	Nome não vazio, opção 0 visível, inválidos tratados.
9. Documentação	PUML atual e guia de execução.	Métodos, visibilidades e relações correspondem ao código.

Regra de avanço

Um marco termina quando você consegue: explicar o conceito, demonstrar o comportamento e mostrar um teste que falharia se a regra fosse quebrada. Código escrito sem essas três evidências ainda está em progresso.

Ritual de cada sessão

- 1 Escolha uma única regra do marco.
- 2 Escreva a previsão e o teste ou observação.
- 3 Faça a menor alteração possível.
- 4 Compile e rode toda a suíte.
- 5 Explique o grafo de objetos resultante.
- 6 Atualize o PUML somente onde o modelo realmente mudou.
- 7 Registre a próxima dúvida; não abra outra frente na mesma sessão.

Planejamento da próxima sessão

- 1 Escolha o primeiro marco ainda não cumprido.
- 2 Quebre-o em três mudanças menores que compilam isoladamente.
- 3 Escreva o teste ou observação de cada mudança antes de programar.
- 4 Defina uma condição de parada para não misturar o marco seguinte.

Critério de conclusão: Seu plano possui tarefas com verbo, objeto e evidência de conclusão.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Marcos reduzem risco e carga cognitiva. 2. Critério de pronto deve existir antes da implementação. 3. Cada passo mantém compilação e testes utilizáveis. 4. Explicação, demonstração e teste formam a conclusão.	1. Qual é o primeiro marco que seu projeto ainda não cumpre? 2. Que prova encerra o marco de transferência? 3. Por que pontuação vem depois do estado persistente?

CAPÍTULO 26

Exercícios em cinco níveis

Os níveis abaixo começam por reconhecimento e terminam em implementação testada. Faça a previsão antes de executar e registre onde seu modelo mental divergiu do resultado.

OBJETIVOS DE APRENDIZAGEM

- ☐ Praticar o mesmo conceito por reconhecimento, previsão e projeto.
- ☐ Identificar lacunas sem depender de copiar uma resposta.
- ☐ Construir evidências objetivas de domínio.

Nível 1 - reconhecer

- 1 Classifique cada nome do projeto como classe, campo, parâmetro ou variável local.
- 2 Encontre todos os membros static e diga a quem pertencem.
- 3 Marque todos os campos com visibilidade de pacote.
- 4 Encontre valores derivados que não precisam ser armazenados.
- 5 Classifique cada seta do PUMML por tipo de relação.

Nível 2 - prever

- 1 Conte quantos objetos Baralho e Carta são criados ao iniciar a versão atual.
- 2 Preveja o valor de jogador.nome criado no inicializador de Blackjack.
- 3 Preveja o naipe de qualquer Carta após o construtor atual.
- 4 Preveja o nome lido depois de nextInt seguido imediatamente por nextLine.
- 5 Preveja o resultado de $A + A + 9 + K$ usando o algoritmo ingênuo e o algoritmo robusto.

Nível 3 - diagnosticar

- 1 Explique por que a lista local de Mao não representa estado persistente.
- 2 Encontre as duas razões independentes pelas quais o if de figuras não funciona.
- 3 Explique por que o teste atual não detecta nenhum bug do jogo.
- 4 Mostre a contradição entre playBlackjack no PUMML e no Java.
- 5 Explique por que um getter de lista poderia quebrar o baralho mesmo com campo private.

Nível 4 - projetar

- 1 Escreva invariantes para todas as classes.
- 2 Desenhe o grafo de objetos após a distribuição inicial.
- 3 Defina contratos de retirada, recebimento e pontuação.
- 4 Compare três modelos para Jogador e Dealer e registre trade-offs.
- 5 Crie uma tabela de resultado completa antes de escrever condicionais.

Nível 5 - implementar e provar

- 1 Implemente um único marco do roteiro.
- 2 Escreva testes que detectem pelo menos dois defeitos plausíveis.
- 3 Faça uma mutação proposital no código e confirme que o teste falha.
- 4 Restaure a regra e execute toda a suíte.
- 5 Atualize o diagrama e explique a mudança sem olhar o código.

Prática de mutação manual

Alterar propositalmente uma regra, como não remover a carta do baralho, é uma forma simples de avaliar o poder do teste. Se tudo continua verde, a suíte não está observando a invariante que você imaginava.

Ficha de registro

Campo	O que escrever
Minha previsão	Estado e saída esperados antes de executar.
O que observei	Valores, erro, teste ou diagrama real.
Onde divergiu	Primeiro ponto em que previsão e realidade se separam.
Conceito faltante	Referência, escopo, tipo, contrato, fluxo ou regra.
Nova prova	Teste pequeno que impedirá a mesma confusão.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Reconhecer não é o mesmo que prever. 2. Previsão escrita torna lacunas visíveis. 3. Projeto antecede implementação em decisões importantes. 4. Teste deve ser desafiado por falha proposital.	1. Em qual nível você explica sem olhar? 2. Que mutação deve quebrar seu primeiro teste real? 3. Como registrar uma divergência de modelo mental?

CAPÍTULO 27

Checklists de revisão do projeto

Checklists não substituem raciocínio, mas evitam esquecer categorias inteiras de erro quando a atenção está concentrada em uma única funcionalidade.

OBJETIVOS DE APRENDIZAGEM

- ☐ Revisar estado, responsabilidades, regras, testes e UML sistematicamente.
- ☐ Usar perguntas em vez de preferências vagas.
- ☐ Criar uma definição de pronto para cada alteração.

Estado e referências

- ☐ Todo valor necessário após um método está em um campo apropriado?
- ☐ Existe objeto criado e imediatamente esquecido?
- ☐ Existe objeto criado e logo substituído sem necessidade?
- ☐ Dois participantes compartilham Mao ou lista sem intenção?
- ☐ Há estado derivado que pode ficar inconsistente?
- ☐ Toda dependência obrigatória é não nula após o construtor?

Responsabilidades e API

- ☐ A classe que executa a operação possui os dados necessários?
- ☐ Carta representa uma única carta?
- ☐ Baralho controla sua coleção sem expô-la para mutação?
- ☐ Mao controla cartas recebidas e pontuação?
- ☐ Blackjack coordena sem virar classe Deus?
- ☐ App permanece livre das regras do domínio?
- ☐ Cada método público expressa ação ou consulta compreensível?

Regras do Blackjack

- ☐ Baralho novo possui 52 combinações únicas?
- ☐ Figuras preservam rank e contribuem 10?
- ☐ Múltiplos ases são ajustados corretamente?
- ☐ Natural é distinguido de qualquer total 21?
- ☐ Estouro é tratado antes da comparação de totais?
- ☐ A variante de soft 17 está declarada?
- ☐ Carta entregue deixa de permanecer entre as cartas restantes?

POO e Java

- ☐ Construtores estabelecem invariantes sem executar toda a partida?
- ☐ Campos internos são privados por decisão consciente?
- ☐ `static` está justificado pela ausência de instância específica?
- ☐ `final` está sendo interpretado sem confundir com imutabilidade profunda?
- ☐ Herança expressa 'é um' e substituíbilidade?
- ☐ Strings de vocabulário fechado foram avaliadas para `enum`?
- ☐ `equals` e `hashCode` estão coerentes quando igualdade lógica é necessária?

Testes e depuração

- ☐ Cada teste pode falhar por um defeito relevante?
- ☐ Cenário, ação e observação estão claros?
- ☐ Há casos de borda e entrada inválida?
- ☐ Aleatoriedade foi controlada ou testada por invariantes?
- ☐ Um bug corrigido ganhou teste de regressão?
- ☐ A primeira causa do erro foi separada das mensagens consequentes?

UML e documentação

- ☐ Existem `@startuml` e `@enduml`?
- ☐ Campos têm nome e tipo; métodos têm parâmetros e retorno?
- ☐ Visibilidades correspondem ao Java?
- ☐ Relações partem de quem conhece a referência?
- ☐ Multiplicidades foram lidas nos dois sentidos?
- ☐ O diagrama está rotulado como atual ou proposto?
- ☐ Métodos e classes não contradizem o código?

Definição de pronto

Uma alteração termina quando compila, preserva invariantes, possui evidência automatizada proporcional ao risco, mantém a interface coerente e deixa o diagrama verdadeiro no nível que ele promete representar.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Checklist organiza atenção, não substitui entendimento. 2. Perguntas objetivas produzem revisão reproduzível. 3. Definição de pronto inclui código, prova e documentação.	1. Que checklist detecta uma mão local? 2. Que pergunta evita herança prematura? 3. Quando uma mudança está realmente pronta?

CAPÍTULO 28

Autoavaliação e próximos desafios

Domínio não é memorizar definições. É prever o comportamento, explicar a responsabilidade e produzir uma prova quando o código muda.

OBJETIVOS DE APRENDIZAGEM

- ☐ Medir domínio por explicação, aplicação e verificação.
- ☐ Escolher extensões que introduzem um conceito novo por vez.
- ☐ Revisitar o diagnóstico inicial e observar progresso real.

Diagnóstico final

Pergunta	Dominei quando consigo...
Qual a diferença entre Mao mao e new Mao()?	Separar tipo, variável, referência e construção.
Por que a mão local desaparece do modelo?	Explicar escopo e alcançabilidade sem dizer que GC é imediato.
Quem deve embaralhar?	Justificar pelo dono do estado e pelo contrato.
Uma dama é 10?	Separar rank, representação e contribuição de pontuação.
Quem decide o ás?	Mostrar que a Mao possui o contexto completo.
new Blackjack joga?	Separar construção de transições da rodada.
static é mais fácil?	Explicar pertencimento à classe e ausência de this.
Um teste passou; está correto?	Avaliar poder de detecção e cobertura de invariantes.

Extensões por conceito

Extensão	Conceitos novos	Pré-requisito
Vários jogadores	Coleção de participantes, iteração, ordem de turno.	Rodada de um jogador completamente testada.
Várias rodadas	Ciclo de vida, reset, sessão versus rodada.	Estado da rodada sem vazamento.
Apostas e fichas	Dinheiro como objeto de valor, transações e invariantes.	Resultado de rodada correto.
Split	Um participante com várias mãos e nova multiplicidade UML.	Mao e turnos bem encapsulados.
Double down	Comandos condicionais e aposta vinculada à mão.	Estado explícito da rodada.
Múltiplos baralhos	Configuração, composição do shoe e contagem total.	Geração por produto cartesiano testada.
Interface gráfica	Arquitetura de apresentação e eventos.	Domínio independente de Scanner.
Simulação estatística	Aleatoriedade controlável, agregação de dados e desempenho.	Partida determinística e testável.

Três rotas de revisão

Rota	Use quando	Capítulos
Fundamentos	Ainda confunde variável, campo, referência e construtor.	2 a 8.
Correção do projeto	Quer continuar o código em pequenos marcos.	12 a 17, 21 a 25.
Revisão rápida	Já estudou e quer recuperar regras.	27, apêndices e cartões.

O triângulo de domínio

Para cada conceito, responda: 1) o que significa? 2) qual erro do meu projeto ele explica? 3) como eu provaria que corrigi? Se uma ponta faltar, revise antes de avançar.

Carta para seu eu futuro

- 1 Explique em uma página como um objeto se lembra de seu estado.
- 2 Desenhe de memória o grafo de objetos da rodada.
- 3 Liste as cinco invariantes mais importantes do projeto.
- 4 Escolha sua próxima extensão e diga qual conceito novo ela exige.

Critério de conclusão: Você consegue reler sua explicação uma semana depois e reconstruir o raciocínio sem o código.

Fechamento do capítulo

Regras para lembrar	Perguntas que você deve responder
1. Domínio combina explicação, aplicação e verificação. 2. Extensões devem entrar sobre uma base estável. 3. Revisão espaçada consolida modelos mentais. 4. Arquitetura é uma decisão justificável, não uma resposta única.	1. Que conceito você ainda não consegue explicar sem olhar? 2. Qual extensão cria a multiplicidade de várias mãos? 3. Que prova mostrará seu próximo progresso?

PARTE 7

Apêndices de consulta

Use esta seção durante a implementação: ela reúne o diagnóstico factual da versão estudada, referências rápidas, glossário, cartões e regras adotadas para a primeira versão.

NESTA PARTE

- Auditoria arquivo a arquivo
- Referência rápida de Java
- Referência rápida de PlantUML
- Glossário essencial
- Cartões com respostas conceituais
- Regras simplificadas da versão 1

CAPÍTULO A

Auditoria da versão estudada

Este retrato corresponde ao código observado em 24 de agosto de 2026. Ele serve como ponto de partida; depois de cada correção, marque o item resolvido e atualize o diagrama.

Estado da compilação

mvn test conclui com sucesso nesta versão. A chamada de Blackjack em App já passa o objeto user. O único teste, porém, continua tautológico e não valida nenhuma regra do jogo.

App.java

Observação	Impacto	Conceito para revisar
nextInt seguido por nextLine	Nome tende a ficar vazio.	Buffer, tokens e linhas.
Laço aceita 0, menu não mostra 0	Saída existe de forma escondida.	Coerência de interface.
Opções 2 e 3 sem cases	Menu promete comportamento inexistente.	Controle de fluxo.
Switch sem break	Futuros cases podem causar fall-through.	Semântica do switch tradicional.
playBlackjack só constrói	Nome do método não corresponde ao efeito.	Construção versus execução.
jogo local não usado	Partida não avança após a construção.	Orquestração explícita.

Blackjack.java

Observação	Impacto	Conceito para revisar
Campos criam objetos antes do construtor	Baralho, jogador e dealer temporários.	Ordem de inicialização.
Jogador temporário recebe nome null	Objeto nasce com estado inválido.	Valores padrão e invariantes.
Construtor substitui os campos	Alocações inúteis; 52 cartas extras.	Criar versus receber dependências.
Campo nome redundante	Nome já pertence ao Jogador.	Fonte única de verdade.
Campos package-private	Estado exposto ao pacote.	Visibilidade e encapsulamento.
embaralhar static e vazio	Comportamento sem dono coerente e sem efeito.	Instância e responsabilidade.

Carta.java

Observação	Impacto	Conceito para revisar
Naípe nunca atribuído	Todas as cartas têm naípe null.	Construtor e invariante.
Gerador envia 1..13	Teste contra J/Q/K não é alcançado.	Representação consistente.
Atribuição após if	Valor calculado é sobrescrito.	Fluxo determinado por chaves.
Carta cria treze cartas	Responsabilidade pouco coesa.	SRP e fábrica.
Campos não final	Imutabilidade não está expressa.	Objeto de valor.
Sem equals/hashCode/toString	Unicidade e saída ficam difíceis.	Igualdade lógica e representação.

Baralho.java

Observação	Impacto	Conceito para revisar
List é campo persistente	Acerto: o estado continua acessível.	Campo versus variável local.
52 objetos são adicionados	Quantidade correta não garante conteúdo válido.	Invariantes compostas.
Quatro Strings de naípe	Erros de digitação permanecem possíveis.	Enum e produto cartesiano.
Sem API de retirada/consulta	Outros objetos não conseguem colaborar encapsuladamente.	Contrato público.
Sem embaralhamento	Ordem nunca é preparada para partida.	Comportamento do dono do estado.
Sem política para vazio	Fluxo futuro fica ambíguo.	Pré-condição e exceção.

Mao, Jogador e Dealer

Arquivo	Observação	Impacto
Mao	ArrayList existe apenas no construtor.	A mão não conserva nenhuma carta.
Jogador	Mao é variável local.	O jogador não possui mão após a construção.
Dealer	Mao é variável local.	O dealer não possui mão após a construção.
Jogador	this.nome = nome está correto.	Exemplo positivo de campo versus parâmetro.
Todos	Imports de ArrayList não usados.	Ruído, não erro de compilação.

blackjack.puml e testes

Área	Observação	Próxima verificação
PUML	Faltam marcadores; membros e métodos contradizem Java.	Renderizar, rotular atual/proposto e revisar um item por vez.
Relações	Intenção geral existe, mas multiplicidades e diamantes pedem rigor.	Ler cada associação nos dois sentidos.
Teste	<code>assertTrue(true)</code> é tautologia.	Criar primeiro teste de estado observável.
Maven	Compilação e teste terminam com código 0.	Não confundir build verde com domínio correto.

CAPÍTULO B

Referência rápida de Java

Fragmentos mínimos para recordar sintaxe e significado. Eles usam exemplos genéricos e não formam uma solução completa do Blackjack.

Classe, campo e construtor

ESTRUTURA BÁSICA

```
public class Exemplo {  
    private final String nome;  
  
    public Exemplo(String nome) {  
        this.nome = nome;  
    }  
}
```

Sintaxe	Significado
private	Acesso restrito à própria classe.
final	Campo recebe valor uma vez durante a construção.
this.nome	Campo da instância atual.
nome	Parâmetro local no construtor.

Métodos

FORMA GERAL

```
visibilidade TipoRetorno nome(Tipo parametro) {  
    // comportamento  
}
```

Forma	Uso
void	Operação não devolve valor explícito.
return valor	Devolve resultado compatível com o tipo declarado.
static	Membro da classe, sem this.
Sobrecarga	Mesmo nome com listas de parâmetros diferentes.

List e ArrayList

OPERAÇÕES ESSENCIAIS

```
List<String> nomes = new ArrayList<>();  
nomes.add("Ana");  
int quantidade = nomes.size();  
String ultimo = nomes.remove(nomes.size() - 1);
```

Pré-condição da remoção

A expressão do último índice só é válida quando a lista não está vazia. A classe deve definir como evitar ou comunicar a tentativa inválida.

Enum

VOCABULÁRIO FECHADO

```
public enum Prioridade {
    BAIXA, MEDIA, ALTA
}

Prioridade p = Prioridade.ALTA;
```

Validação

EXEMPLO GENÉRICO

```
if (texto == null || texto.isBlank()) {
    throw new IllegalArgumentException(
        "texto obrigatório"
    );
}
```

Exceção	Significado típico
IllegalArgumentException	O chamador forneceu argumento inválido.
IllegalStateException	A operação não é permitida no estado atual.
NullPointerException	Uma referência null foi usada onde se esperava objeto.
InputMismatchException	Scanner encontrou token incompatível com o tipo solicitado.

Comparações

Tipo	Comparação apropriada
Primitivos	== compara valor.
Referências	== compara identidade da referência.
String	equals compara conteúdo.
Enum	== é apropriado para constantes enum.
Objeto de valor	equals/hashCode implementam igualdade lógica.

CAPÍTULO C

Referência rápida de PlantUML

Use este apêndice como folha de consulta durante a atualização do diagrama. Cada símbolo deve continuar acompanhado de uma decisão de domínio.

Classe completa genérica

MODELO SINTÁTICO

```
@startuml
title Modelo de exemplo

class Proprietario {
    - item: Item
    + executar(valor: int): boolean
}

class Item {
    - codigo: String
}

Proprietario "1" --> "1" Item : conhece
@enduml
```

Visibilidade

Símbolo	Java
+	public
-	private
#	protected
~	package-private

Relações

SÍMBOLOS ESSENCIAIS

```
A ..> B      ' dependência
A --> B      ' associação navegável
A o-- B      ' agregação
A *-- B      ' composição
A --|> B     ' herança
A ..|> B     ' implementação de interface
```

Multiplicidades

EXEMPLOS DE CARDINALIDADE

```
A "1" --> "0..*" B
A "0..1" --> "1..*" C
```

Sequência

MODELO GENÉRICO

```
@startuml
actor Usuario
Usuario -> Interface : ação
Interface -> Dominio : comando()
Dominio --> Interface : resultado
@enduml
```

Estados

MODELO GENÉRICO

```
@startuml
[*] --> Criado
Criado --> Preparado : preparar
Preparado --> Encerrado : concluir
Encerrado --> [*]
@enduml
```

Checklist de renderização

Renderize depois de cada pequena alteração. Confirme que texto não virou membro acidental, que setas não se cruzam excessivamente e que o diagrama continua legível sem ampliar.

CAPÍTULO D

Glossário essencial

Definições curtas para consulta. Volte ao capítulo indicado quando uma palavra estiver sendo usada sem clareza.

Termo	Definição
Abstração	Representação que destaca propriedades relevantes e omite detalhes desnecessários.
Acoplamento	Quantidade e intensidade de conhecimento entre partes do sistema.
Alcançabilidade	Existência de um caminho de referências a partir de raízes ativas até um objeto.
Aliasing	Duas ou mais referências apontando para o mesmo objeto mutável.
API	Conjunto de operações expostas para outros componentes.
Associação	Relação estrutural em que um objeto conhece outro.
Atributo / campo	Estado armazenado em uma instância ou classe.
Classe	Tipo que define estrutura e comportamento de instâncias.
Classe Deus	Classe que concentra responsabilidades demais e conhece detalhes excessivos.
Coesão	Grau em que os elementos de uma classe servem ao mesmo propósito.
Coleção	Estrutura que agrupa referências ou valores.
Composição	Relação todo-parte forte, geralmente exclusiva e ligada ao ciclo de vida.
Construtor	Rotina usada para inicializar uma nova instância válida.
Contrato	Expectativas de entrada, efeito, retorno e falha de uma operação.
Cópia defensiva	Nova estrutura criada para impedir mutação externa da estrutura interna.
Dependência	Uso de outro tipo ou objeto necessário para realizar uma tarefa.
Domínio	Conjunto de conceitos e regras do problema que o software representa.
Encapsulamento	Controle do estado por meio de uma API que preserva invariantes.
Enum	Tipo com conjunto fechado de constantes nomeadas.
Escopo	Região do código em que um nome pode ser usado.
Estado	Conjunto atual de valores mantidos por um objeto.
Estado derivado	Informação calculável a partir de outra fonte de verdade.
Garbage collector	Mecanismo que recupera memória de objetos inalcançáveis em momento não determinado.
Generic	Parâmetro de tipo que aumenta segurança e reutilização, como List.
Herança	Relação de generalização entre subtipo e tipo mais geral.
Identidade	Propriedade de ser exatamente a mesma instância.
Imutabilidade	Impossibilidade de alterar o estado observável após a construção.
Injeção de dependência	Fornecimento de uma dependência a um objeto, geralmente por construtor ou método.
Instância	Objeto concreto criado a partir de um tipo.

Termo	Definição
Invariante	Condição que deve permanecer verdadeira durante a vida válida de um objeto.
Máquina de estados	Modelo de fases e transições permitidas.
Mutabilidade	Capacidade de alterar estado após a criação.
Navegabilidade	Direção em que uma relação pode ser percorrida por referências.
Objeto	Entidade de execução com identidade, estado e comportamento.
Objeto de valor	Objeto definido principalmente pela igualdade de seus dados.
Orquestração	Coordenação de uma sequência de colaborações entre objetos.
Parâmetro	Variável local que recebe um argumento durante uma chamada.
Polimorfismo	Uso de diferentes implementações por meio de um contrato comum.
Pré-condição	Condição esperada antes de uma operação.
Pós-condição	Condição garantida após uma operação bem-sucedida.
Referência	Valor usado para alcançar uma instância de objeto.
Refatoração	Mudança de estrutura que preserva comportamento observável.
Responsabilidade	Papel coerente que uma classe desempenha no modelo.
Sombreamento	Nome local ou parâmetro que oculta outro nome, como um campo.
Substituibilidade	Capacidade de usar um subtipo onde o tipo geral é esperado sem quebrar contratos.
Teste de regressão	Teste que impede o retorno de um defeito já corrigido.
Variável local	Nome temporário cujo escopo está dentro de método, construtor ou bloco.
Visibilidade	Regra de quem pode acessar um membro.

CAPÍTULO E

Cartões de revisão e respostas conceituais

Cubra a coluna de resposta, tente explicar em voz alta e somente depois confira. As respostas são conceituais; nenhuma entrega a implementação completa.

Pergunta	Resposta conceitual
O que um objeto se lembra?	Do estado alcançável por seus campos; variáveis locais não viram memória do objeto.
Mao mao cria uma mão?	Não. Declara uma variável de referência. A construção ocorre com new Mao().
Quando uma variável local sai de escopo, o objeto morre?	Não necessariamente. Ele fica elegível para GC somente se nenhuma referência alcançável permanecer.
O que this significa?	A instância atual em um contexto de instância.
Java passa objetos por referência?	Java passa tudo por valor; para objetos, copia o valor da referência.
private garante encapsulamento?	Não sozinho. A API também precisa impedir alterações que violem invariantes.
final List é imutável?	Não. final impede reatribuir o campo; a lista ainda pode mudar.
Quem deve embaralhar?	Naturalmente o objeto que possui a coleção e pode preservar suas invariantes: o baralho.
static significa global?	Não. Significa membro da classe, sem uma instância this.
Uma Carta cria um naipe?	Pode tecnicamente ter fábrica static, mas a responsabilidade de montar o conjunto é mais coesa em Baralho ou fábrica.
Por que usar enum para naipe?	O conjunto é fechado e o compilador deve rejeitar valores fora dele.
Dama é dez?	Dama é o rank; dez é sua contribuição de pontuação no Blackjack.
Quem interpreta o ás?	Mao, pois possui o contexto das outras cartas.
O que é mão macia?	Mão em que pelo menos um ás ainda conta como 11.
Todo 21 é blackjack natural?	Não. Natural exige exatamente duas cartas, normalmente ás e uma carta de valor 10.
addAll clona cartas?	Não. Copia referências para os mesmos objetos.
O que prova List?	Que os elementos inseridos por código tipado são compatíveis com Carta; não prova unicidade nem validade.
== e equals são iguais?	Não. == compara identidade de referências; equals pode expressar igualdade lógica.
Por que new Blackjack não joga?	new constrói a instância; outras operações precisam iniciar as transições.
O que um teste assertTrue(true) prova?	Apenas que o método de teste executou; não prova regra do domínio.
Como testar shuffle?	Verificar preservação de tamanho e conjunto ou controlar a aleatoriedade; não exigir ordem sempre diferente.

Pergunta	Resposta conceitual
Quando usar herança?	Quando existe relação 'é um', contrato comum e substituíbilidade, não apenas para remover duplicação.
O que 0..* significa?	Zero ou muitas instâncias no extremo indicado da associação.
O que é fonte única de verdade?	Um único estado principal do qual informações derivadas são calculadas.
Como saber se terminei um marco?	Consgo explicar, demonstrar e testar o comportamento definido.

CAPÍTULO F

Regras simplificadas adotadas para a versão 1

Blackjack possui variações. Para evitar ambiguidade durante o aprendizado, esta especificação reduzida define um cenário coerente antes de adicionar apostas, split e regras de cassino.

Decisão	Regra adotada
Participantes	Um jogador humano e um dealer.
Baralho	Um baralho padrão de 52 cartas, sem curingas.
Distribuição	Duas rodadas; jogador recebe uma, dealer recebe uma, repetindo uma vez.
Objetivo	Obter total mais próximo de 21 sem ultrapassar.
Cartas numéricas	Valem o número impresso.
Figuras	J, Q e K valem 10 e preservam seu rank.
Ás	Vale 11 quando possível sem estourar; caso necessário, passa a contribuir 1.
Natural	A + carta de valor 10 nas duas primeiras cartas.
Jogador	Pode pedir cartas ou parar; estourar encerra sua chance de vitória.
Dealer	Compra até alcançar 17 e para em qualquer 17, inclusive soft 17, nesta versão.
Empate	Totais válidos iguais produzem push.
Fora do escopo	Apostas, split, double down, seguro, surrender e múltiplos jogadores.

Tabela de decisão resumida

Prioridade	Condição	Resultado
1	Jogador estourou	Dealer vence.
2	Dealer estourou e jogador não	Jogador vence.
3	Um possui natural e o outro não	Natural vence.
4	Totais válidos diferentes	Maior total vence.
5	Totais válidos iguais	Empate / push.

Por que declarar a variante

A política de soft 17 e pagamentos de natural variam em regras reais. Um software precisa registrar a regra escolhida; caso contrário, código e testes podem estar individualmente coerentes e ainda discordar entre si.

Folha final de compromisso

- ☐ Consigo desenhar o grafo de objetos sem consultar o código.
- ☐ Consigo explicar referência, escopo, this e construtor com minhas palavras.
- ☐ Consigo justificar cada responsabilidade principal.
- ☐ Consigo calcular qualquer combinação de ases apresentada neste guia.
- ☐ Consigo escrever um teste que falha por uma regra quebrada.
- ☐ Consigo distinguir o modelo atual do modelo proposto no PUMML.
- ☐ Consigo implementar o próximo marco sem copiar uma solução completa.

Três ideias para levar

ESTADO	RESPONSABILIDADE	PROVA
Um objeto só recorda o que permanece alcançável por seus campos.	O comportamento deve ficar perto do estado e da regra que o justificam.	Toda decisão importante precisa de observação capaz de detectar sua quebra.

Plano da próxima sessão

Preencha antes de abrir o editor	Sua resposta
Marco em que estou	_____
Uma única invariante	_____
Teste que deve falhar primeiro	_____
Condição de parada	_____

Ideia final: orientação a objetos não começa com herança. Começa quando você consegue responder onde o estado vive, quem pode modificá-lo, que objeto possui contexto para uma regra e como provar que a colaboração preservou as invariantes.